Source folder: /Users/jonat/Documents/GitHub/master_thesis
Files in this part: 17

File: federated_learning_in_mobile/ios/Pods/Local Podspecs/path_provider_foundation.podspec.json

---

```
{
  "name": "path_provider_foundation",
  "version": "0.0.1",
  "summary": "An iOS and macOS implementation of the path_provider plugin.",
  "description": "An iOS and macOS implementation of the Flutter plugin for getting commonly used locations
 on the filesystem.",
  "homepage": "https://github.com/flutter/packages/tree/main/packages/path_provider/path_provider_foundatio
n",
  "license": {
    "type": "BSD",
    "file": "../LICENSE"
  },
  "authors": {
    "Flutter Dev Team": "flutter-dev@googlegroups.com"
  },
  "source": {
    "http": "https://github.com/flutter/packages/tree/main/packages/path_provider/path_provider_foundation"
  },
  "source_files": "path_provider_foundation/Sources/path_provider_foundation/**/*.swift",
  "ios": {
    "dependencies": {
      "Flutter": []
    },
    "xcconfig": {
      "LIBRARY_SEARCH_PATHS": "$(TOOLCHAIN_DIR)/usr/lib/swift/$(PLATFORM_NAME)/ $(SDKROOT)/usr/lib/swift",
      "LD_RUNPATH_SEARCH_PATHS": "/usr/lib/swift"
    }
  },
  "osx": {
    "dependencies": {
      "FlutterMacOS": []
    }
  },
  "platforms": {
    "ios": "13.0",
    "osx": "10.15"
  },
  "swift_versions": "5.0",
  "resource_bundles": {
    "path_provider_foundation_privacy": [
      "path_provider_foundation/Sources/path_provider_foundation/Resources/PrivacyInfo.xcprivacy"
    ]
  },
  "swift_version": "5.0"
}
```

File: federated_learning_in_mobile/ios/Pods/Local Podspecs/share_plus.podspec.json

---

```
{
  "name": "share_plus",
  "version": "0.0.1",
  "summary": "Flutter Share",
  "description": "A Flutter plugin to share content from your Flutter app via the platform's share dialog.\
nDownloaded by pub (not CocoaPods).",
  "homepage": "https://github.com/fluttercommunity/plus_plugins",
  "license": {
    "type": "BSD",
    "file": "../LICENSE"
  },
  "authors": {
    "Flutter Community Team": "authors@fluttercommunity.dev"
  },
  "source": {
    "http": "https://github.com/fluttercommunity/plus_plugins/tree/main/packages/share_plus/share_plus"
  },
  "documentation_url": "https://pub.dev/packages/share_plus",
  "source_files": "Classes/**/*",
  "public_header_files": "Classes/**/*.h",
  "dependencies": {
```

```
    "Flutter": []
  },
  "ios": {
    "weak_frameworks": "LinkPresentation"
  },
  "platforms": {
    "ios": "11.0"
  },
  "pod_target_xcconfig": {
    "DEFINES_MODULE": "YES"
  }
}
```

File: federated_learning_in_mobile/ios/Runner/Assets.xcassets/AppIcon.appiconset/Contents.json

---

```
{
  "images" : [
    {
      "size" : "20x20",
      "idiom" : "iphone",
      "filename" : "Icon-App-20x20@2x.png",
      "scale" : "2x"
    },
    {
      "size" : "20x20",
      "idiom" : "iphone",
      "filename" : "Icon-App-20x20@3x.png",
      "scale" : "3x"
    },
    {
      "size" : "29x29",
      "idiom" : "iphone",
      "filename" : "Icon-App-29x29@1x.png",
      "scale" : "1x"
    },
    {
      "size" : "29x29",
      "idiom" : "iphone",
      "filename" : "Icon-App-29x29@2x.png",
      "scale" : "2x"
    },
    {
      "size" : "29x29",
      "idiom" : "iphone",
      "filename" : "Icon-App-29x29@3x.png",
      "scale" : "3x"
    },
    {
      "size" : "40x40",
      "idiom" : "iphone",
      "filename" : "Icon-App-40x40@2x.png",
      "scale" : "2x"
    },
    {
      "size" : "40x40",
      "idiom" : "iphone",
      "filename" : "Icon-App-40x40@3x.png",
      "scale" : "3x"
    },
    {
      "size" : "60x60",
      "idiom" : "iphone",
      "filename" : "Icon-App-60x60@2x.png",
      "scale" : "2x"
    },
    {
      "size" : "60x60",
      "idiom" : "iphone",
      "filename" : "Icon-App-60x60@3x.png",
      "scale" : "3x"
    },
    {
      "size" : "20x20",
      "idiom" : "ipad",
```

```
      "filename" : "Icon-App-20x20@1x.png",
      "scale" : "1x"
    },
    {
      "size" : "20x20",
      "idiom" : "ipad",
      "filename" : "Icon-App-20x20@2x.png",
      "scale" : "2x"
    },
    {
      "size" : "29x29",
      "idiom" : "ipad",
      "filename" : "Icon-App-29x29@1x.png",
      "scale" : "1x"
    },
    {
      "size" : "29x29",
      "idiom" : "ipad",
      "filename" : "Icon-App-29x29@2x.png",
      "scale" : "2x"
    },
    {
      "size" : "40x40",
      "idiom" : "ipad",
      "filename" : "Icon-App-40x40@1x.png",
      "scale" : "1x"
    },
    {
      "size" : "40x40",
      "idiom" : "ipad",
      "filename" : "Icon-App-40x40@2x.png",
      "scale" : "2x"
    },
    {
      "size" : "76x76",
      "idiom" : "ipad",
      "filename" : "Icon-App-76x76@1x.png",
      "scale" : "1x"
    },
    {
      "size" : "76x76",
      "idiom" : "ipad",
      "filename" : "Icon-App-76x76@2x.png",
      "scale" : "2x"
    },
    {
      "size" : "83.5x83.5",
      "idiom" : "ipad",
      "filename" : "Icon-App-83.5x83.5@2x.png",
      "scale" : "2x"
    },
    {
      "size" : "1024x1024",
      "idiom" : "ios-marketing",
      "filename" : "Icon-App-1024x1024@1x.png",
      "scale" : "1x"
    }
  ],
  "info" : {
    "version" : 1,
    "author" : "xcode"
  }
}
```

File: federated_learning_in_mobile/ios/Runner/Assets.xcassets/LaunchImage.imageset/Contents.json

---

```
{
  "images" : [
    {
      "idiom" : "universal",
      "filename" : "LaunchImage.png",
      "scale" : "1x"
    },
    {
```

```
      "idiom" : "universal",
      "filename" : "LaunchImage@2x.png",
      "scale" : "2x"
    },
    {
      "idiom" : "universal",
      "filename" : "LaunchImage@3x.png",
      "scale" : "3x"
    }
  ],
  "info" : {
    "version" : 1,
    "author" : "xcode"
  }
}
```

File: federated_learning_in_mobile/ios/Runner/Assets.xcassets/LaunchImage.imageset/README.md

```
# Launch Screen Assets

You can customize the launch screen with your own desired assets by replacing the image files in this direc
tory.

You can also do it by opening your Flutter project's Xcode project with `open ios/Runner.xcworkspace`, sele
cting `Runner/Assets.xcassets` in the Project Navigator and dropping in the desired images.
```

File: federated_learning_in_mobile/lib/main.dart

```
import 'package:flutter/material.dart';
import 'services/fl_client.dart';
import 'services/api_client.dart';
import 'utils/metrics_collector.dart';
import 'screens/recommendations_page.dart';
import 'package:device_info_plus/device_info_plus.dart';
import 'package:share_plus/share_plus.dart';

void main() {
  runApp(const FederatedLearningApp());
}

class FederatedLearningApp extends StatelessWidget {
  const FederatedLearningApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Federated Learning Client',
      theme: ThemeData(
        colorScheme: ColorScheme.dark(
          surface: const Color(0xFF212D3F),
          primary: const Color(0xFF00BCD4),
          secondary: const Color(0xFF00BCD4),
        ),
        scaffoldBackgroundColor: const Color(0xFF1A2332),
        cardTheme: const CardThemeData(
          color: Color(0xFF212D3F),
          elevation: 0,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.all(Radius.circular(12)),
          ),
        ),
        useMaterial3: true,
      ),
      home: const FederatedLearningHomePage(),
    );
  }
}

class FederatedLearningHomePage extends StatefulWidget {
  const FederatedLearningHomePage({super.key});

  @override
  State<FederatedLearningHomePage> createState() =>
      _FederatedLearningHomePageState();
```

```
}

class _FederatedLearningHomePageState extends State<FederatedLearningHomePage> {
  final TextEditingController _serverUrlController = TextEditingController(
    text:
        '', // User must enter their PC's IP address (e.g., http://192.168.x.x:8000)
  );
  final TextEditingController _clientIdController = TextEditingController(
    text: 'android_client_${DateTime.now().millisecondsSinceEpoch % 10000}',
  );

  FederatedLearningClient? _client;
  bool _isConnected = false;
  bool _isTraining = false;
  String _status = 'Ready';
  final List<String> _logs = [];
  MetricsCollector? _metricsCollector;
  int _trainingRound = 0;
  String? _deviceId;

  @override
  void dispose() {
    _serverUrlController.dispose();
    _clientIdController.dispose();
    super.dispose();
  }

  void _addLog(String message) {
    final timestamp = DateTime.now().toString().substring(11, 19);
    final logMessage = '[$timestamp] $message';
    // Debug print to console
    debugPrint('[FL_CLIENT_LOG] $logMessage');
    print(
      '[FL_CLIENT_LOG] $logMessage',
    ); // Also use print for better visibility
    setState(() {
      _logs.add(logMessage);
      if (_logs.length > 100) {
        _logs.removeAt(0);
      }
    });
  }

  Future<void> _connectToServer() async {
    final serverUrl = _serverUrlController.text.trim();

    if (serverUrl.isEmpty) {
      _showError(
        'Please enter server URL (e.g., http://192.168.1.100:8000)\n\nOn your PC, run: ipconfig\nto find yo
ur IP address',
      );
      return;
    }

    // Validate URL format
    if (!serverUrl.startsWith('http://') && !serverUrl.startsWith('https://')) {
      _showError('Server URL must start with http:// or https://');
      return;
    }

    // Parse and validate the host address
    try {
      final uri = Uri.parse(serverUrl);
      final host = uri.host.toLowerCase();

      // Check for invalid addresses
      if (host == 'localhost' ||
          host == '192.168.29.147' ||
          host == '127.0.0.1' ||
          host == '0.0.0.0' ||
          host.isEmpty) {
        String errorMsg = 'Invalid server address: $host\n\n';
        if (host == '0.0.0.0') {
          errorMsg += '0.0.0.0 is not a valid address to connect to!\n\n';
```

```
        } else {
          errorMsg +=
              'localhost/127.0.0.1 will not work from mobile device!\n\n';
        }
        errorMsg += 'Use your PC\'s actual IP address instead:\n';
        errorMsg += '• Windows: Run "ipconfig" and look for "IPv4 Address"\n';
        errorMsg += '• Mac/Linux: Run "ifconfig" or "ip addr"\n';
        errorMsg += '• Example: http://192.168.1.100:8000';
        _showError(errorMsg);
        return;
      }

      // Warn if port is missing
      if (uri.port == 0) {
        _showError('Server URL must include a port number (e.g., :8000)');
        return;
      }

      _addLog('NETWORK: DNS lookup resolved for coordinator.');
    } catch (e) {
      _showError(
        'Invalid URL format: $e\n\nPlease use format: http://IP_ADDRESS:PORT\nExample: http://192.168.1.100
:8000',
      );
      return;
    }

    setState(() {
      _status = 'Connecting...';
      _isTraining = true;
    });

    try {
      final apiClient = ApiClient(serverUrl: serverUrl);
      await apiClient.healthCheck();

      _addLog('INFO: Handshake protocol version 3.2.0.');

      // Create client
      _client = FederatedLearningClient(
        clientId: _clientIdController.text,
        serverUrl: serverUrl,
        numUsers: 943, // Will be updated after fetching model
        numItems: 1682, // Will be updated after fetching model
        embeddingDim: 16,
      );

      await _client!.initialize();

      // Get device info for metrics collection
      try {
        final deviceInfo = DeviceInfoPlugin();
        if (Theme.of(context).platform == TargetPlatform.android) {
          final androidInfo = await deviceInfo.androidInfo;
          _deviceId = '${androidInfo.brand}_${androidInfo.model}'.replaceAll(
            ' ',
            '_',
          );
        } else {
          _deviceId = 'device_${DateTime.now().millisecondsSinceEpoch}';
        }
      } catch (e) {
        _deviceId = 'device_${DateTime.now().millisecondsSinceEpoch}';
        _addLog('Warning: Could not get device info: $e');
      }

      // Initialize metrics collector (non-blocking - continue even if it fails)
      String? experimentId;
      try {
        experimentId = createExperimentId(
          embeddingDim: _client!.embeddingDim,
          numClients: 1,
          deviceId: _deviceId,
        );
```

```
      _metricsCollector = MetricsCollector(experimentId: experimentId);
      await _metricsCollector!.initialize();
      _addLog('Metrics collector initialized');

      // Set experiment config
      _metricsCollector!.setConfig({
        'num_users': _client!.numUsers,
        'num_items': _client!.numItems,
        'embedding_dim': _client!.embeddingDim,
        'client_id': _clientIdController.text,
        'device_id': _deviceId,
        'server_url': serverUrl,
      });
    } catch (e) {
      _addLog('Warning: Metrics collector initialization failed: $e');
      _addLog('Training will continue but results won\'t be saved');
      // Continue anyway - don't block connection
    }

    setState(() {
      _isConnected = true;
      _status = 'Connected';
      _isTraining = false;
    });

    _addLog('SYSTEM: Client initialized successfully.');
    if (experimentId != null) {
      _addLog('Experiment ID: $experimentId');
      _addLog('Metrics will be saved after training');
    }

    // Try to fetch initial model (optional - model might not be initialized yet)
    try {
      await _client!.fetchGlobalModel();
      _addLog('SYSTEM: Local model weight buffer allocated (512MB).');
    } catch (e) {
      _addLog('WARN: Model not initialized on server yet.');
      _addLog('INFO: You can fetch model later when starting training.');
    }
  } catch (e, stackTrace) {
    setState(() {
      _status = 'Connection failed';
      _isTraining = false;
    });
    debugPrint('[FL_CLIENT_ERROR] Connection error: $e');
    debugPrint('[FL_CLIENT_ERROR] Stack trace: $stackTrace');
    print('[FL_CLIENT_ERROR] Connection error: $e');
    print('[FL_CLIENT_ERROR] Stack trace: $stackTrace');

    // Provide helpful error message based on error type
    String errorMessage = 'Failed to connect to server.\n\n';

    if (e.toString().contains('Connection refused') ||
        e.toString().contains('SocketException')) {
      errorMessage += 'Connection was refused. This usually means:\n';
      errorMessage += '1. Server is not running\n';
      errorMessage += '2. Wrong IP address or port\n';
      errorMessage += '3. Firewall blocking connection\n';
      errorMessage += '4. Device not on same network\n\n';
      errorMessage += 'Troubleshooting:\n';
      errorMessage += '• Verify server is running on your PC\n';
      errorMessage +=
          '• Check IP address: Run "ipconfig" (Windows) or "ifconfig" (Mac/Linux)\n';
      errorMessage += '• Ensure both devices are on the same Wi-Fi network\n';
      errorMessage += '• Try disabling firewall temporarily\n';
      errorMessage += '• Check server logs for errors';
    } else if (e.toString().contains('timeout') ||
        e.toString().contains('TimeoutException')) {
      errorMessage += 'Connection timed out. Check:\n';
      errorMessage += '• Server is running and accessible\n';
      errorMessage += '• Network connection is stable\n';
      errorMessage += '• Firewall is not blocking the connection';
    } else {
```

```
        errorMessage += 'Error: $e';
      }

      _addLog('Error: $e');
      _showError(errorMessage);
    }
  }

  Future<void> _runTrainingRound() async {
    if (_client == null || !_isConnected) {
      _showError('Please connect to server first');
      return;
    }

    setState(() {
      _isTraining = true;
      _status = 'Training Round ${_trainingRound + 1}...';
    });

    try {
      // First, make sure we have the global model (it updates numUsers and numItems)
      if (_trainingRound == 0) {
        _addLog('Fetching global model...');
        try {
          await _client!.fetchGlobalModel();
          _addLog(
            'Global model fetched. Model: ${_client!.numUsers} users, ${_client!.numItems} items',
          );
        } catch (e) {
          _addLog('Warning: Could not fetch model, using current dimensions');
        }

        // Load some sample data (in real app, this would come from device)
        // For demo, create minimal sample data
        if (_client!.localData.isEmpty) {
          // Generate sample data based on ACTUAL model dimensions (updated after fetchGlobalModel)
          final sampleData = <List<dynamic>>[];
          final random = DateTime.now().millisecondsSinceEpoch;
          final numUsers = _client!.numUsers;
          final numItems = _client!.numItems;

          _addLog(
            'Generating sample data for $numUsers users, $numItems items...',
          );

          for (int i = 0; i < 10; i++) {
            final userId = (random + i) % numUsers;
            final itemId = (random + i * 2) % numItems;

            // Validate IDs are in range (just to be safe)
            if (userId >= 0 &&
                userId < numUsers &&
                itemId >= 0 &&
                itemId < numItems) {
              sampleData.add([
                userId,
                itemId,
                1.0, // Binarized rating
              ]);
            } else {
              _addLog(
                'Warning: Skipping invalid sample: user=$userId (max=$numUsers), item=$itemId (max=$numItem
s)',
              );
            }
          }
          _client!.loadLocalData(sampleData);
          _addLog('Loaded ${sampleData.length} sample interactions');
        }
      }

      _addLog('Starting training round ${_trainingRound + 1}...');

      // Run training round
```

```
final metrics = await _client!.runTrainingRound();

_trainingRound++;

// Collect metrics
if (_metricsCollector != null) {
  final trainMetrics = metrics['train'] as Map<String, dynamic>? ?? {};
  final testMetrics = metrics['test'] as Map<String, dynamic>? ?? {};
  final uploadMetrics = metrics['upload'] as Map<String, dynamic>? ?? {};
  final resourceMetrics =
      uploadMetrics['resource_metrics'] as Map<String, dynamic>? ?? {};

  _metricsCollector!.addRoundMetrics(
    roundNum: _trainingRound - 1, // Use round number before increment
    trainLoss: (trainMetrics['loss'] as num?)?.toDouble() ?? 0.0,
    testMetrics: testMetrics,
    aggregationInfo: {
      'num_clients': 1,
      'total_samples': trainMetrics['samples'] ?? 0,
    },
    clientMetrics: [
      {
        'client_id': _clientIdController.text,
        'loss': trainMetrics['loss'] ?? 0.0,
        'samples': trainMetrics['samples'] ?? 0,
        'epochs': trainMetrics['epochs'] ?? 0,
      },
    ],
    resourceMetrics: resourceMetrics,
  );

  // Save results after each round (both locally and to server)
  try {
    // Save locally first
    final jsonPath = await _metricsCollector!.saveJson();
    final csvPath = await _metricsCollector!.saveCsvSummary();

    _addLog('Results saved locally:');
    _addLog('  JSON: ${jsonPath.split('/').last}');
    _addLog('  CSV: ${csvPath.split('/').last}');

    // Also upload to server (saves to PC automatically!)
    await _uploadResultsToServer();
  } catch (e) {
    _addLog('Warning: Failed to save results: $e');
  }
}

setState(() {
  _status = 'Round ${_trainingRound - 1} complete';
  _isTraining = false;
});

_addLog('Training round ${_trainingRound - 1} complete');
_addLog(
  'Loss: ${metrics['train']?['loss']?.toStringAsFixed(4) ?? "N/A"}',
);
_addLog(
  'Accuracy: ${metrics['test']?['accuracy']?.toStringAsFixed(4) ?? "N/A"}',
);

// Show success message after 10 rounds
if (_trainingRound > 10) {
  _addLog('■ All 10 rounds complete!');
  _addLog('Results have been uploaded to your PC automatically.');

  // Calculate and add final metrics
  _calculateFinalMetrics();

  // Show dialog after final round
  Future.delayed(const Duration(seconds: 1), () {
    _uploadResultsToServer(showSuccessDialog: true);
  });
}
```

```
    } catch (e, stackTrace) {
      setState(() {
        _status = 'Training failed';
        _isTraining = false;
      });
      debugPrint('[FL_CLIENT_ERROR] Training error: $e');
      debugPrint('[FL_CLIENT_ERROR] Stack trace: $stackTrace');
      print('[FL_CLIENT_ERROR] Training error: $e');
      print('[FL_CLIENT_ERROR] Stack trace: $stackTrace');
      _addLog('Training error: $e');
      _showError('Training failed: $e');
    }
  }

  void _ensureSampleDataLoaded() {
    if (_client == null || _client!.localData.isNotEmpty) {
      return;
    }

    final sampleData = <List<dynamic>>[];
    final seed = DateTime.now().millisecondsSinceEpoch;
    final numUsers = _client!.numUsers;
    final numItems = _client!.numItems;
    final sampleCount = numItems > 0 ? 15 : 0;

    for (int i = 0; i < sampleCount; i++) {
      final userId = (seed + i) % numUsers;
      final itemId = (seed + i * 3) % numItems;
      sampleData.add([userId, itemId, 1.0]);
    }

    _client!.loadLocalData(sampleData);
    _addLog('Demo data loaded: ${sampleData.length} interactions');
  }

  Future<void> _runDemoMode() async {
    if (_client == null || !_isConnected) {
      _showError('Please connect to server first');
      return;
    }

    if (_isTraining) {
      _showError('Training is already in progress');
      return;
    }

    setState(() {
      _status = 'Demo mode: preparing model...';
    });

    try {
      await _client!.fetchGlobalModel();
      _ensureSampleDataLoaded();
      _addLog('Demo mode: running one training round...');
      await _runTrainingRound();

      if (!mounted) {
        return;
      }

      if (_client!.model != null) {
        _addLog('Demo mode: opening recommendations screen.');
        _openRecommendationsScreen();
      }
    } catch (e) {
      _showError('Demo mode failed: $e');
    }
  }

  /// Run 10 rounds automatically
  Future<void> _runAll10Rounds() async {
    if (_client == null || !_isConnected) {
      _showError('Please connect to server first');
      return;
```

```
    }

    if (_isTraining) {
      _showError('Training already in progress');
      return;
    }

    // Reset round counter
    _trainingRound = 0;

    // Confirm action
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Run 10 Training Rounds'),
        content: const Text(
          'This will automatically run 10 training rounds.\n\n'
          'The training will continue until all 10 rounds are complete.\n'
          'You can stop by closing the app if needed.\n\n'
          'Proceed?',
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.of(context).pop(false),
            child: const Text('Cancel'),
          ),
          ElevatedButton(
            onPressed: () => Navigator.of(context).pop(true),
            child: const Text('Start'),
          ),
        ],
      ),
    );

    if (confirmed != true) {
      return;
    }

    setState(() {
      _status = 'Running 10 rounds...';
    });

    _addLog('Starting automatic 10-round training...');

    // Run 10 rounds sequentially
    for (int round = 1; round <= 10; round++) {
      if (!_isConnected || _client == null) {
        _addLog('Training stopped: connection lost');
        break;
      }

      _addLog('=== Round $round/10 ===');
      await _runTrainingRound();

      // Small delay between rounds
      await Future.delayed(const Duration(milliseconds: 500));
    }

    if (_trainingRound > 10) {
      setState(() {
        _status = 'All 10 rounds complete!';
      });
    }
  }

  /// Calculate final metrics after all rounds are complete
  void _calculateFinalMetrics() {
    if (_metricsCollector == null) return;

    final rounds = _metricsCollector!.experimentData['rounds'] as List;
    if (rounds.isEmpty) return;

    // Get metrics from last round
    final lastRound = rounds.last as Map<String, dynamic>;
```

```
    final lastTestMetrics =
        lastRound['test_metrics'] as Map<String, dynamic>? ?? {};

    // Calculate averages across all rounds
    double avgAccuracy = 0.0;
    double avgLoss = 0.0;
    double avgHit10 = 0.0;
    double avgNDCG10 = 0.0;
    int count = 0;

    for (var round in rounds) {
      final roundMap = round as Map<String, dynamic>;
      final testMetrics =
          roundMap['test_metrics'] as Map<String, dynamic>? ?? {};

      if (testMetrics['accuracy'] != null) {
        avgAccuracy += (testMetrics['accuracy'] as num).toDouble();
        avgLoss += (roundMap['train_loss'] as num?)?.toDouble() ?? 0.0;
        avgHit10 += (testMetrics['Hit@10'] as num?)?.toDouble() ?? 0.0;
        avgNDCG10 += (testMetrics['NDCG@10'] as num?)?.toDouble() ?? 0.0;
        count++;
      }
    }

    if (count > 0) {
      avgAccuracy /= count;
      avgLoss /= count;
      avgHit10 /= count;
      avgNDCG10 /= count;
    }

    // Set final metrics
    _metricsCollector!.addFinalMetrics({
      'accuracy': lastTestMetrics['accuracy'] ?? avgAccuracy,
      'mse': lastTestMetrics['mse'] ?? 0.0,
      'mae': lastTestMetrics['mae'] ?? 0.0,
      'Hit@10': lastTestMetrics['Hit@10'] ?? avgHit10,
      'NDCG@10': lastTestMetrics['NDCG@10'] ?? avgNDCG10,
      'Precision@10': lastTestMetrics['Precision@10'] ?? 0.0,
      'Recall@10': lastTestMetrics['Recall@10'] ?? 0.0,
      'final_train_loss': avgLoss,
      'avg_accuracy': avgAccuracy,
      'avg_Hit@10': avgHit10,
      'avg_NDCG@10': avgNDCG10,
    });

    // Add client metrics summary
    _metricsCollector!.addClientMetrics(_clientIdController.text, {
      'total_rounds': rounds.length,
      'total_samples':
          (rounds.last as Map)['aggregation']?['total_samples'] ?? 0,
      'final_loss': avgLoss,
      'final_accuracy': avgAccuracy,
    });

    // Add mobile metrics summary
    double totalBatteryDrain = 0.0;
    int totalTimeMs = 0;
    for (var round in rounds) {
      final roundMap = round as Map<String, dynamic>;
      final resourceMetrics =
          roundMap['resource_metrics'] as Map<String, dynamic>? ?? {};
      totalBatteryDrain +=
          (resourceMetrics['battery_drain'] as num?)?.toDouble() ?? 0.0;
      totalTimeMs +=
          (resourceMetrics['training_time_ms'] as num?)?.toInt() ?? 0;
    }

    _metricsCollector!.addMobileMetrics(_deviceId ?? 'unknown', {
      'total_rounds': rounds.length,
      'total_battery_drain': totalBatteryDrain,
      'total_training_time_ms': totalTimeMs,
      'avg_battery_drain_per_round': rounds.isNotEmpty
          ? totalBatteryDrain / rounds.length
```

```
        : 0.0,
    'avg_training_time_ms_per_round': rounds.isNotEmpty
        ? totalTimeMs / rounds.length
        : 0.0,
  });

  _addLog('Final metrics calculated and saved');
}

void _showError(String message) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text(message), backgroundColor: Colors.red),
  );
}

Future<void> _showResultsLocation() async {
  if (_metricsCollector == null) return;

  try {
    await _metricsCollector!.initialize();
    final resultsPath = _metricsCollector!.resultsPath;
    final resultFiles = await _metricsCollector!.getResultFiles();

    showDialog(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Results Location'),
        content: SingleChildScrollView(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            mainAxisSize: MainAxisSize.min,
            children: [
              const Text(
                'Results are saved to Downloads folder:',
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 8),
              SelectableText(
                resultsPath,
                style: const TextStyle(fontFamily: 'monospace', fontSize: 12),
              ),
              const SizedBox(height: 16),
              const Text(
                'Files:',
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 8),
              if (resultFiles.isEmpty)
                const Text(
                  'No files saved yet',
                  style: TextStyle(fontStyle: FontStyle.italic),
                )
              else
                ...resultFiles.map(
                  (f) => Text(
                    '• ${f.path.split('/').last}',
                    style: const TextStyle(fontSize: 12),
                  ),
                ),
              const SizedBox(height: 16),
              Text(
                'Training rounds: $_trainingRound',
                style: const TextStyle(fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 16),
              const Text(
                'How to access files:',
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 8),
              const Text(
                '1. Open Android File Manager\n'
                '2. Go to Downloads folder\n'
                '3. Look for "FL_Results" folder\n'
```

```
                '4. Your JSON and CSV files are inside\n\n'
                'Or use ADB:\n'
                'adb pull /storage/emulated/0/Download/FL_Results/',
                style: TextStyle(fontSize: 12),
              ),
            ),
          ],
        ),
      ),
      actions: [
        if (resultFiles.isNotEmpty)
          TextButton.icon(
            onPressed: () => _shareResults(),
            icon: const Icon(Icons.share),
            label: const Text('Share Files'),
          ),
        TextButton(
          onPressed: () => Navigator.of(context).pop(),
          child: const Text('Close'),
        ),
      ],
    ),
  );
 } catch (e) {
  _showError('Failed to get results location: $e');
 }
}

Future<void> _shareResults() async {
 if (_metricsCollector == null) return;

 try {
  final resultFiles = await _metricsCollector!.getResultFiles();
  if (resultFiles.isEmpty) {
   _showError('No result files to share');
   return;
  }

  // Share the most recent JSON file
  final jsonFiles = resultFiles
      .where((f) => f.path.endsWith('.json'))
      .toList();
  if (jsonFiles.isEmpty) {
   _showError('No JSON files to share');
   return;
  }

  // Sort by modification time (newest first)
  jsonFiles.sort(
   (a, b) => b.lastModifiedSync().compareTo(a.lastModifiedSync()),
  );
  final latestFile = jsonFiles.first;

  final xFile = XFile(latestFile.path, mimeType: 'application/json');
  await Share.shareXFiles(
   [xFile],
   text: 'Federated Learning Experiment Results',
   subject: 'FL Results: ${_metricsCollector!.experimentId}',
  );

  _addLog('Shared file: ${latestFile.path.split('/').last}');
 } catch (e) {
  _showError('Failed to share files: $e');
 }
}

/// Upload results to server (saves directly to PC)
Future<void> _uploadResultsToServer({bool showSuccessDialog = false}) async {
 if (_client == null || !_isConnected) {
  _showError('Please connect to server first');
  return;
 }

 if (_metricsCollector == null) {
  _showError('No results to upload. Run training rounds first.');
```

```
  return;
}

setState(() {
  _status = 'Uploading results to PC...';
});

try {
  final experimentData = _metricsCollector!.getExperimentData();

  _addLog('Uploading results to server (PC)...');
  _addLog('Experiment ID: ${_metricsCollector!.experimentId}');

  final uploadResponse = await _client!.apiClient.uploadMobileResults(
    experimentId: _metricsCollector!.experimentId,
    experimentData: experimentData,
  );

  final jsonPath =
      uploadResponse['json_file'] as String? ??
      'mobile_results/${_metricsCollector!.experimentId}.json';
  final csvPath = uploadResponse['csv_file'] as String?;
  final message =
      uploadResponse['message'] as String? ?? 'Results saved to PC';

  _addLog('✓ Results uploaded successfully!');
  _addLog('  Saved on PC: $message');
  _addLog('  JSON: $jsonPath');
  if (csvPath != null) {
    _addLog('  CSV: $csvPath');
  }

  setState(() {
    _status = 'Results uploaded to PC';
  });

  if (showSuccessDialog) {
    showDialog(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('✓ Results Uploaded to PC'),
        content: SingleChildScrollView(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            mainAxisSize: MainAxisSize.min,
            children: [
              const Text(
                'Your results have been saved directly to your PC!',
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 16),
              const Text(
                'Location on your PC:',
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 8),
              SelectableText(
                jsonPath,
                style: const TextStyle(
                  fontFamily: 'monospace',
                  fontSize: 12,
                ),
              ),
              if (csvPath != null) ...[
                const SizedBox(height: 8),
                SelectableText(
                  csvPath,
                  style: const TextStyle(
                    fontFamily: 'monospace',
                    fontSize: 12,
                  ),
                ),
              ],
              const SizedBox(height: 16),
```

```
                const Text(
                  'How to access:',
                  style: TextStyle(fontWeight: FontWeight.bold),
                ),
                const SizedBox(height: 8),
                const Text(
                  '1. Go to your project folder\n'
                  '2. Open the "mobile_results" folder\n'
                  '3. Find your JSON and CSV files\n\n'
                  'Or check your server terminal - it shows the full path!',
                  style: TextStyle(fontSize: 12),
                ),
              ],
            ),
          ),
          actions: [
            TextButton(
              onPressed: () => Navigator.of(context).pop(),
              child: const Text('OK'),
            ),
          ],
        ),
      );
    }
  } catch (uploadError) {
    _addLog('✗ Upload failed: $uploadError');
    setState(() {
      _status = 'Upload failed';
    });

    String errorMsg = 'Failed to upload results to PC.\n\n';
    if (uploadError.toString().contains('Connection refused') ||
        uploadError.toString().contains('SocketException')) {
      errorMsg += 'Cannot connect to server.\n';
      errorMsg += '• Make sure server is running\n';
      errorMsg += '• Check server URL is correct\n';
      errorMsg += '• Verify both devices are on same network';
    } else {
      errorMsg += 'Error: $uploadError';
    }

    _showError(errorMsg);
  }
}

void _openRecommendationsScreen() {
  if (_client == null || !_isConnected) {
    _showError('Please connect to server first.');
    return;
  }

  if (_client!.model == null) {
    _showError('No trained model loaded yet. Run at least one round first.');
    return;
  }

  Navigator.of(context).push(
    MaterialPageRoute(builder: (_) => RecommendationsPage(client: _client!)),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFF1A2332),
    body: SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(20.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            // Header with title and settings icon
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
```

```
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text(
              'Federated Client',
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: TextStyle(
                fontSize: 32,
                fontWeight: FontWeight.bold,
                color: Colors.white,
              ),
            ),
            const SizedBox(height: 4),
            Text(
              'Researcher Edition v2.4',
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: TextStyle(fontSize: 16, color: Colors.grey[400]),
            ),
          ],
        ),
      ),
      const SizedBox(width: 12),
      Container(
        decoration: BoxDecoration(
          color: const Color(0xFF212D3F),
          borderRadius: BorderRadius.circular(12),
          border: Border.all(
            color: const Color(0xFF00BCD4).withValues(alpha: 0.3),
            width: 1,
          ),
        ),
        child: Row(
          mainAxisSize: MainAxisSize.min,
          children: [
            IconButton(
              tooltip: 'Open recommendations',
              icon: const Icon(
                Icons.movie,
                color: Color(0xFF00BCD4),
              ),
              onPressed: (_isConnected && _client?.model != null)
                  ? _openRecommendationsScreen
                  : null,
            ),
            IconButton(
              icon: const Icon(
                Icons.settings,
                color: Color(0xFF00BCD4),
              ),
              onPressed: () {
                // Settings action - can be implemented later
                ScaffoldMessenger.of(context).showSnackBar(
                  const SnackBar(
                    content: Text('Settings coming soon'),
                  ),
                );
              },
            ),
          ],
        ),
      ),
    ],
  ),

  const SizedBox(height: 24),

  // Connection Section
  Card(
    child: Padding(
```

```
padding: const EdgeInsets.all(20.0),
child: Column(
  crossAxisAlignment: CrossAxisAlignment.stretch,
  children: [
    Row(
      children: [
        Icon(
          Icons.dns,
          color: const Color(0xFF00BCD4),
          size: 20,
        ),
        const SizedBox(width: 8),
        const Text(
          'SERVER CONFIGURATION',
          style: TextStyle(
            fontSize: 13,
            fontWeight: FontWeight.w600,
            letterSpacing: 0.5,
            color: Colors.grey,
          ),
        ),
      ],
    ),
    const SizedBox(height: 20),
    Text(
      'SERVER URL',
      style: TextStyle(
        fontSize: 12,
        fontWeight: FontWeight.w600,
        color: const Color(0xFF00BCD4),
        letterSpacing: 0.5,
      ),
    ),
    const SizedBox(height: 8),
    TextField(
      controller: _serverUrlController,
      decoration: InputDecoration(
        hintText: 'wss://fl-coordinator.research.net',
        hintStyle: TextStyle(color: Colors.grey[600]),
        filled: true,
        fillColor: const Color(0xFF1A2332),
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(8),
          borderSide: BorderSide.none,
        ),
        contentPadding: const EdgeInsets.symmetric(
          horizontal: 16,
          vertical: 14,
        ),
      ),
      style: const TextStyle(
        color: Colors.white70,
        fontSize: 15,
      ),
      enabled: !_isConnected,
      keyboardType: TextInputType.url,
    ),
    const SizedBox(height: 16),
    Text(
      'CLIENT ID',
      style: TextStyle(
        fontSize: 12,
        fontWeight: FontWeight.w600,
        color: Colors.grey[400],
        letterSpacing: 0.5,
      ),
    ),
    const SizedBox(height: 8),
    TextField(
      controller: _clientIdController,
      decoration: InputDecoration(
        hintText: 'android_client_6149',
        hintStyle: TextStyle(color: Colors.grey[600]),
        filled: true,
```

```
                fillColor: const Color(0xFF1A2332),
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(8),
                  borderSide: BorderSide.none,
                ),
                contentPadding: const EdgeInsets.symmetric(
                  horizontal: 16,
                  vertical: 14,
                ),
              ),
              style: const TextStyle(
                color: Colors.white70,
                fontSize: 15,
              ),
              enabled: !_isConnected,
            ),
            const SizedBox(height: 20),
            SizedBox(
              height: 54,
              child: ElevatedButton(
                onPressed: _isConnected ? null : _connectToServer,
                style: ElevatedButton.styleFrom(
                  backgroundColor: const Color(0xFF00BCD4),
                  foregroundColor: Colors.black87,
                  disabledBackgroundColor: Colors.grey[700],
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(8),
                  ),
                  elevation: 0,
                ),
                child: Row(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    const Icon(Icons.flash_on, size: 20),
                    const SizedBox(width: 8),
                    const Text(
                      'CONNECT TO SERVER',
                      style: TextStyle(
                        fontSize: 15,
                        fontWeight: FontWeight.bold,
                        letterSpacing: 0.5,
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ],
        ),
      ),
    ),

    const SizedBox(height: 16),

    // Status Section
    Card(
      child: Padding(
        padding: const EdgeInsets.all(20.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                Expanded(
                  child: Row(
                    children: [
                      Container(
                        width: 12,
                        height: 12,
                        decoration: BoxDecoration(
                          shape: BoxShape.circle,
                          color: _isConnected
                              ? const Color(0xFF4CAF50)
```

```
                    : Colors.grey,
              ),
            ),
            const SizedBox(width: 10),
            Expanded(
              child: Text(
                'Status: $_status',
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
                style: const TextStyle(
                  fontSize: 18,
                  fontWeight: FontWeight.w600,
                  color: Colors.white,
                ),
              ),
            ),
          ],
        ),
      ),
      const SizedBox(width: 8),
      Container(
        padding: const EdgeInsets.symmetric(
          horizontal: 12,
          vertical: 6,
        ),
        decoration: BoxDecoration(
          color: const Color(0xFF1A2332),
          borderRadius: BorderRadius.circular(6),
        ),
        child: Text(
          _isTraining ? 'TRAINING' : 'IDLE',
          style: const TextStyle(
            fontSize: 12,
            fontWeight: FontWeight.bold,
            color: Color(0xFF4CAF50),
            letterSpacing: 0.5,
          ),
        ),
      ),
    ],
  ),
  const SizedBox(height: 20),
  Row(
    children: [
      Expanded(
        child: SizedBox(
          height: 100,
          child: ElevatedButton(
            onPressed: (_isConnected && !_isTraining)
                ? _runTrainingRound
                : null,
            style: ElevatedButton.styleFrom(
              backgroundColor: const Color(0xFF2A3A4F),
              foregroundColor: Colors.white70,
              disabledBackgroundColor: const Color(
                0xFF1E2936,
              ),
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(12),
              ),
              elevation: 0,
            ),
            child: const Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Icon(
                  Icons.play_arrow,
                  size: 32,
                  color: Color(0xFF00BCD4),
                ),
                SizedBox(height: 8),
                Text(
                  'Run 1 Round',
                  style: TextStyle(
```

```
                    fontSize: 16,
                    fontWeight: FontWeight.w600,
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
      const SizedBox(width: 12),
      Expanded(
        child: SizedBox(
          height: 100,
          child: ElevatedButton(
            onPressed: (_isConnected && !_isTraining)
                ? _runAll10Rounds
                : null,
            style: ElevatedButton.styleFrom(
              backgroundColor: const Color(0xFF2A3A4F),
              foregroundColor: Colors.white70,
              disabledBackgroundColor: const Color(
                0xFF1E2936,
              ),
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(12),
              ),
              elevation: 0,
            ),
            child: const Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Icon(
                  Icons.fast_forward,
                  size: 32,
                  color: Color(0xFF00BCD4),
                ),
                SizedBox(height: 8),
                Text(
                  'Run 10 Rounds',
                  style: TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w600,
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    ],
  ),
  const SizedBox(height: 12),
  SizedBox(
    height: 52,
    child: ElevatedButton.icon(
      onPressed: (_isConnected && !_isTraining)
          ? _runDemoMode
          : null,
      icon: const Icon(
        Icons.present_to_all,
        color: Color(0xFF00BCD4),
      ),
      label: const Text(
        'Demo Mode: Train and Show Recommendations',
        style: TextStyle(fontWeight: FontWeight.w600),
      ),
      style: ElevatedButton.styleFrom(
        backgroundColor: const Color(0xFF243447),
        foregroundColor: Colors.white,
        disabledBackgroundColor: const Color(0xFF1E2936),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(10),
        ),
        elevation: 0,
```

```
              ),
            ),
          ),
          if (_isTraining) ...[
            const SizedBox(height: 8),
            const Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                SizedBox(
                  width: 16,
                  height: 16,
                  child: CircularProgressIndicator(strokeWidth: 2),
                ),
                SizedBox(width: 8),
                Text('Training...'),
              ],
            ),
          ],
          if (_metricsCollector != null && _trainingRound > 0) ...[
            const SizedBox(height: 12),
            Row(
              children: [
                Expanded(
                  child: OutlinedButton.icon(
                    onPressed: (_isConnected && !_isTraining)
                        ? _uploadResultsToServer
                        : null,
                    icon: const Icon(Icons.cloud_upload, size: 18),
                    label: const Text(
                      'Upload to PC',
                      style: TextStyle(fontSize: 13),
                    ),
                    style: OutlinedButton.styleFrom(
                      foregroundColor: const Color(0xFF00BCD4),
                      side: const BorderSide(
                        color: Color(0xFF00BCD4),
                      ),
                      padding: const EdgeInsets.symmetric(
                        vertical: 12,
                      ),
                    ),
                  ),
                ),
                const SizedBox(width: 8),
                Expanded(
                  child: OutlinedButton.icon(
                    onPressed: _showResultsLocation,
                    icon: const Icon(Icons.folder, size: 18),
                    label: const Text(
                      'Location',
                      style: TextStyle(fontSize: 13),
                    ),
                    style: OutlinedButton.styleFrom(
                      foregroundColor: Colors.grey[400],
                      side: BorderSide(color: Colors.grey[700]!),
                      padding: const EdgeInsets.symmetric(
                        vertical: 12,
                      ),
                    ),
                  ),
                ),
              ],
            ),
          ],
        ],
      ),
    ),
  ),
), // End of Status Section Card
// Logs Section
const SizedBox(height: 16),
Card(
  child: Padding(
    padding: const EdgeInsets.all(20.0),
    child: Column(
```

```
crossAxisAlignment: CrossAxisAlignment.stretch,
children: [
  Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Row(
        children: [
          Icon(
            Icons.live_tv,
            color: Colors.grey[400],
            size: 20,
          ),
          const SizedBox(width: 8),
          const Text(
            'LIVE LOGS',
            style: TextStyle(
              fontSize: 13,
              fontWeight: FontWeight.w600,
              letterSpacing: 0.5,
              color: Colors.grey,
            ),
          ),
        ],
      ),
      TextButton(
        onPressed: () {
          setState(() {
            _logs.clear();
          });
        },
        style: TextButton.styleFrom(
          foregroundColor: Colors.grey[500],
          padding: const EdgeInsets.symmetric(
            horizontal: 12,
            vertical: 8,
          ),
        ),
        child: const Text(
          'CLEAR CONSOLE',
          style: TextStyle(
            fontSize: 11,
            fontWeight: FontWeight.w600,
            letterSpacing: 0.5,
          ),
        ),
      ),
    ],
  ),
  const SizedBox(height: 12),
  Container(
    height: 250,
    padding: const EdgeInsets.all(12),
    decoration: BoxDecoration(
      color: const Color(0xFF0D1117),
      borderRadius: BorderRadius.circular(8),
    ),
    child: _logs.isEmpty
        ? Center(
            child: Text(
              'No logs yet. Connect to server to see activity.',
              style: TextStyle(
                color: Colors.grey[600],
                fontSize: 13,
                fontStyle: FontStyle.italic,
              ),
            ),
          )
        : ListView.builder(
            reverse: false,
            itemCount: _logs.length,
            itemBuilder: (context, index) {
              final log = _logs[index];
              Color logColor = const Color(
                0xFF58A6FF,
```

```
                                    ); // Default blue

                                    // Color code logs based on type
                                    if (log.contains('SYSTEM:')) {
                                      logColor = const Color(0xFF4CAF50); // Green
                                    } else if (log.contains('NETWORK:')) {
                                      logColor = const Color(0xFF00BCD4); // Cyan
                                    } else if (log.contains('INFO:')) {
                                      logColor = const Color(0xFF58A6FF); // Blue
                                    } else if (log.contains('WARN:')) {
                                      logColor = const Color(
                                        0xFFFFA726,
                                      ); // Orange
                                    } else if (log.contains('ERROR:')) {
                                      logColor = const Color(0xFFEF5350); // Red
                                    }

                                    return Padding(
                                      padding: const EdgeInsets.symmetric(
                                        vertical: 2.0,
                                      ),
                                      child: Text(
                                        log,
                                        style: TextStyle(
                                          color: logColor,
                                          fontFamily: 'Courier',
                                          fontSize: 12,
                                          height: 1.4,
                                        ),
                                      ),
                                    );
                                  },
                                ),
                            ), // End of Container
                          ],
                        ),
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ),
        ),
      );
  }
}
```

File: federated_learning_in_mobile/lib/models/matrix_factorization.dart

```
/// Matrix Factorization model for movie recommendations
/// Pure Dart implementation compatible with PyTorch server model
library;

class MatrixFactorization {
  final int numUsers;
  final int numItems;
  final int embeddingDim;

  // Embedding matrices: [num_users x embedding_dim], [num_items x embedding_dim]
  late List<List<double>> userEmbeddings;
  late List<List<double>> itemEmbeddings;

  MatrixFactorization({
    required this.numUsers,
    required this.numItems,
    required this.embeddingDim,
  }) {
    // Initialize embeddings with small random values
    userEmbeddings = _initializeEmbeddings(numUsers, embeddingDim);
    itemEmbeddings = _initializeEmbeddings(numItems, embeddingDim);
  }

  /// Initialize embedding matrix with small random values (similar to PyTorch)
  List<List<double>> _initializeEmbeddings(int size, int dim) {
    final embeddings = <List<double>>[];
```

```
    final random = DateTime.now().millisecondsSinceEpoch; // Simple seed

    for (int i = 0; i < size; i++) {
      final row = <double>[];
      for (int j = 0; j < dim; j++) {
        // Generate random value between -0.01 and 0.01
        final value = ((random + i * dim + j) % 2000 - 1000) / 100000.0;
        row.add(value);
      }
      embeddings.add(row);
    }
    return embeddings;
  }

  /// Forward pass: predict rating for user-item pair
  double predict(int userId, int itemId) {
    if (userId >= numUsers || itemId >= numItems || userId < 0 || itemId < 0) {
      throw ArgumentError('Invalid user_id or item_id');
    }

    final userEmb = userEmbeddings[userId];
    final itemEmb = itemEmbeddings[itemId];

    // Dot product: sum(user_emb[i] * item_emb[i])
    double result = 0.0;
    for (int i = 0; i < embeddingDim; i++) {
      result += userEmb[i] * itemEmb[i];
    }

    return result;
  }

  /// Predict ratings for multiple user-item pairs (batch prediction)
  List<double> predictBatch(List<int> userIds, List<int> itemIds) {
    if (userIds.length != itemIds.length) {
      throw ArgumentError('userIds and itemIds must have same length');
    }

    final predictions = <double>[];
    for (int i = 0; i < userIds.length; i++) {
      predictions.add(predict(userIds[i], itemIds[i]));
    }
    return predictions;
  }

  /// Get model state dictionary (for serialization)
  Map<String, dynamic> getStateDict() {
    return {
      'user_embedding.weight': userEmbeddings,
      'item_embedding.weight': itemEmbeddings,
    };
  }

  /// Load model state from dictionary (for deserialization)
  void loadStateDict(Map<String, dynamic> stateDict) {
    userEmbeddings = List<List<double>>.from(
      stateDict['user_embedding.weight'].map((row) =>
        List<double>.from(row.map((val) => val.toDouble()))
      )
    );
    itemEmbeddings = List<List<double>>.from(
      stateDict['item_embedding.weight'].map((row) =>
        List<double>.from(row.map((val) => val.toDouble()))
      )
    );
  }

  /// Get all trainable parameters (flattened for gradient computation)
  List<List<List<double>>> getParameters() {
    return [userEmbeddings, itemEmbeddings];
  }

  /// Update parameters (used during training)
  void updateParameters(List<List<double>> userGrad, List<List<double>> itemGrad, double learningRate) {
```

```
    // Gradient descent update
    for (int i = 0; i < numUsers; i++) {
      for (int j = 0; j < embeddingDim; j++) {
        userEmbeddings[i][j] -= learningRate * userGrad[i][j];
      }
    }

    for (int i = 0; i < numItems; i++) {
      for (int j = 0; j < embeddingDim; j++) {
        itemEmbeddings[i][j] -= learningRate * itemGrad[i][j];
      }
    }
  }
}
```

File: federated_learning_in_mobile/lib/screens/recommendations_page.dart

---

```
import 'package:flutter/material.dart';

import '../services/fl_client.dart';
import '../services/movie_title_service.dart';
import '../services/recommendation_service.dart';

class RecommendationsPage extends StatefulWidget {
  final FederatedLearningClient client;

  const RecommendationsPage({super.key, required this.client});

  @override
  State<RecommendationsPage> createState() => _RecommendationsPageState();
}

class _RecommendationsPageState extends State<RecommendationsPage> {
  final RecommendationService _recommendationService =
      const RecommendationService();
  final MovieTitleService _movieTitleService = const MovieTitleService();

  List<int> _candidateUsers = const <int>[];
  List<MovieRecommendation> _recommendations = const <MovieRecommendation>[];
  Map<int, String> _movieTitles = const <int, String>{};
  int? _selectedUserId;
  int _topN = 10;
  bool _excludeSeenItems = true;
  bool _isLoadingTitles = true;
  final Set<int> _submittingSeenItems = <int>{};
  String? _error;

  @override
  void initState() {
    super.initState();
    _initialize();
  }

  Future<void> _initialize() async {
    try {
      final titles = await _movieTitleService.loadTitles();
      if (!mounted) {
        return;
      }
      _movieTitles = titles;
    } catch (_) {
      // Fallback is handled in RecommendationService with Movie #id labels.
      _movieTitles = const <int, String>{};
    }

    _candidateUsers = _recommendationService.getCandidateUsers(widget.client);
    if (_candidateUsers.isNotEmpty) {
      _selectedUserId = _candidateUsers.first;
      _refreshRecommendations();
    } else {
      _error =
          'No users available. Connect and run at least one training round first.';
    }
```

```
    if (mounted) {
      setState(() {
        _isLoadingTitles = false;
      });
    }
  }

  void _refreshRecommendations() {
    if (_selectedUserId == null) {
      return;
    }

    try {
      final results = _recommendationService.topNRecommendations(
        client: widget.client,
        userId: _selectedUserId!,
        topN: _topN,
        movieTitles: _movieTitles,
        excludeSeenItems: _excludeSeenItems,
      );

      setState(() {
        _error = null;
        _recommendations = results;
      });
    } catch (e) {
      setState(() {
        _recommendations = const <MovieRecommendation>[];
        _error = 'Failed to generate recommendations: $e';
      });
    }
  }

  Future<void> _markAsSeen(MovieRecommendation rec, int index) async {
    final userId = _selectedUserId;
    if (userId == null ||
        rec.isSeen ||
        _submittingSeenItems.contains(rec.itemId)) {
      return;
    }

    setState(() {
      _submittingSeenItems.add(rec.itemId);
      _recommendations[index] = rec.copyWith(
        isSeen: true,
        explanation:
            'Predicted score ${rec.score.toStringAsFixed(4)} - Seen before',
      );
    });

    try {
      await widget.client.markRecommendationSeen(
        userId: userId,
        itemId: rec.itemId,
        rating: 1.0,
      );

      if (!mounted) {
        return;
      }
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text('Marked "${rec.title}" as seen')));
    } catch (e) {
      if (!mounted) {
        return;
      }
      setState(() {
        _recommendations[index] = rec;
      });
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('Failed to send seen event: $e'),
```

```
        backgroundColor: Colors.redAccent,
      ),
    );
  } finally {
    if (mounted) {
      setState(() {
        _submittingSeenItems.remove(rec.itemId);
      });
    }
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Movie Recommendations'),
      actions: [
        IconButton(
          tooltip: 'Refresh',
          onPressed: _refreshRecommendations,
          icon: const Icon(Icons.refresh),
        ),
      ],
    ),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          _buildControls(),
          const SizedBox(height: 12),
          if (_isLoadingTitles)
            const Text('Loading movie titles...')
          else if (_error != null)
            Text(_error!, style: const TextStyle(color: Colors.redAccent))
          else
            Text(
              'Showing ${_recommendations.length} recommendations for user $_selectedUserId',
              style: Theme.of(context).textTheme.bodyMedium,
            ),
          const SizedBox(height: 12),
          Expanded(
            child: _recommendations.isEmpty
                ? const Center(child: Text('No recommendations yet'))
                : ListView.separated(
                    itemCount: _recommendations.length,
                    separatorBuilder: (_, __) => const Divider(height: 1),
                    itemBuilder: (context, index) {
                      final rec = _recommendations[index];
                      final isSubmitting = _submittingSeenItems.contains(
                        rec.itemId,
                      );
                      return Padding(
                        padding: const EdgeInsets.symmetric(
                          horizontal: 12,
                          vertical: 10,
                        ),
                        child: Row(
                          crossAxisAlignment: CrossAxisAlignment.start,
                          children: [
                            CircleAvatar(child: Text('${index + 1}')),
                            const SizedBox(width: 12),
                            Expanded(
                              child: Column(
                                crossAxisAlignment: CrossAxisAlignment.start,
                                mainAxisSize: MainAxisSize.min,
                                children: [
                                  Text(
                                    rec.title,
                                    style: Theme.of(context)
                                        .textTheme
                                        .titleMedium,
                                  ),
```

```
                                const SizedBox(height: 4),
                                Text(rec.explanation),
                              ],
                            ),
                          ),
                          const SizedBox(width: 10),
                          Column(
                            mainAxisSize: MainAxisSize.min,
                            crossAxisAlignment: CrossAxisAlignment.end,
                            children: [
                              Container(
                                padding: const EdgeInsets.symmetric(
                                  horizontal: 8,
                                  vertical: 2,
                                ),
                                decoration: BoxDecoration(
                                  color: rec.isSeen
                                      ? Colors.orange.withValues(alpha: 0.15)
                                      : Colors.green.withValues(alpha: 0.15),
                                  borderRadius: BorderRadius.circular(8),
                                  border: Border.all(
                                    color: rec.isSeen
                                        ? Colors.orange
                                        : Colors.green,
                                  ),
                                ),
                                child: Text(
                                  rec.isSeen ? 'Seen' : 'Unseen',
                                  style: TextStyle(
                                    color: rec.isSeen
                                        ? Colors.orange
                                        : Colors.green,
                                    fontSize: 11,
                                    fontWeight: FontWeight.w600,
                                  ),
                                ),
                              ),
                              const SizedBox(height: 6),
                              FilledButton.tonal(
                                style: FilledButton.styleFrom(
                                  minimumSize: const Size(0, 28),
                                  tapTargetSize:
                                      MaterialTapTargetSize.shrinkWrap,
                                  visualDensity: VisualDensity.compact,
                                  padding: const EdgeInsets.symmetric(
                                    horizontal: 10,
                                    vertical: 6,
                                  ),
                                ),
                                onPressed: (rec.isSeen || isSubmitting)
                                    ? null
                                    : () => _markAsSeen(rec, index),
                                child: isSubmitting
                                    ? const SizedBox(
                                        height: 12,
                                        width: 12,
                                        child: CircularProgressIndicator(
                                          strokeWidth: 2,
                                        ),
                                      )
                                    : Text(rec.isSeen ? 'Seen' : 'Mark seen'),
                              ),
                            ],
                          ),
                        ],
                      ),
                    ],
                  ),
                );
              },
            ),
          ),
        ],
      ),
    ),
  );
```

```
}

Widget _buildControls() {
  final dropdownItems = _candidateUsers
      .map(
        (userId) =>
            DropdownMenuItem<int>(value: userId, child: Text('User $userId')),
      )
      .toList();

  return Card(
    child: Padding(
      padding: const EdgeInsets.all(12),
      child: Column(
        children: [
          Row(
            children: [
              const Text('User:'),
              const SizedBox(width: 12),
              Expanded(
                child: DropdownButton<int>(
                  value: _selectedUserId,
                  isExpanded: true,
                  items: dropdownItems,
                  onChanged: (value) {
                    if (value == null) {
                      return;
                    }
                    setState(() {
                      _selectedUserId = value;
                    });
                    _refreshRecommendations();
                  },
                ),
              ),
            ],
          ),
          const SizedBox(height: 8),
          Row(
            children: [
              const Text('Top N:'),
              Expanded(
                child: Slider(
                  min: 5,
                  max: 30,
                  divisions: 5,
                  value: _topN.toDouble(),
                  label: _topN.toString(),
                  onChanged: (value) {
                    setState(() {
                      _topN = value.round();
                    });
                  },
                  onChangeEnd: (_) => _refreshRecommendations(),
                ),
              ),
              Text(_topN.toString()),
            ],
          ),
          SwitchListTile(
            contentPadding: EdgeInsets.zero,
            title: const Text('Exclude seen items'),
            value: _excludeSeenItems,
            onChanged: (value) {
              setState(() {
                _excludeSeenItems = value;
              });
              _refreshRecommendations();
            },
          ),
        ],
      ),
    ),
  );
```

```
  }
}
```

File: federated_learning_in_mobile/lib/services/api_client.dart

```
import 'dart:convert';
import 'dart:io';
import 'package:http/http.dart' as http;

/// API client for communicating with the federated learning server
class ApiClient {
  final String _baseUrl;

  ApiClient({required String serverUrl})
    : _baseUrl = serverUrl.endsWith('/')
          ? serverUrl.substring(0, serverUrl.length - 1)
          : serverUrl;

  /// Get normalized base URL (without trailing slash)
  String get baseUrl => _baseUrl;

  /// Check if server is available
  Future<Map<String, dynamic>> healthCheck() async {
    final url = Uri.parse('$_baseUrl/healthz');
    print('[API_CLIENT] Health check: GET $url');
    print('[API_CLIENT] Parsed host: ${url.host}, port: ${url.port}');

    // Validate host before attempting connection
    if (url.host.isEmpty || url.host == '0.0.0.0') {
      throw Exception(
        'Invalid server address: ${url.host}. Use your PC\'s actual IP address (e.g., 192.168.1.100)',
      );
    }

    try {
      final response = await http
          .get(url)
          .timeout(
            const Duration(seconds: 10),
            onTimeout: () {
              throw Exception(
                'Connection timeout: Server did not respond within 10 seconds. Check if server is running a
nd accessible.',
              );
            },
          );
      print('[API_CLIENT] Health check response: ${response.statusCode}');
      print('[API_CLIENT] Health check headers: ${response.headers}');
      print('[API_CLIENT] Health check body (raw): ${response.body}');
      print('[API_CLIENT] Health check body length: ${response.body.length}');

      if (response.statusCode == 200) {
        try {
          final decoded = jsonDecode(response.body) as Map<String, dynamic>;
          print('[API_CLIENT] Health check parsed successfully: $decoded');
          return decoded;
        } catch (jsonError) {
          print('[API_CLIENT_ERROR] JSON decode failed: $jsonError');
          print(
            '[API_CLIENT_ERROR] Response body that failed to parse: ${response.body}',
          );
          throw Exception(
            'Server returned invalid JSON. Expected JSON but got: ${response.body.substring(0, 100)}...',
          );
        }
      } else {
        throw Exception(
          'Server health check failed: ${response.statusCode} - ${response.body}',
        );
      }
    } on SocketException catch (e) {
      print('[API_CLIENT_ERROR] Socket error: $e');
      throw Exception(
        'Cannot connect to server at ${url.host}:${url.port}. '
```

```
        'Check that:\n'
        '1. Server is running\n'
        '2. IP address is correct\n'
        '3. Port is correct\n'
        '4. Both devices are on the same network',
      );
    } catch (e, stackTrace) {
      print('[API_CLIENT_ERROR] Health check failed: $e');
      print('[API_CLIENT_ERROR] Stack trace: $stackTrace');
      rethrow;
    }
  }

  /// Register client with server
  Future<Map<String, dynamic>> register(String clientId) async {
    final url = Uri.parse('$_baseUrl/register');
    print('[API_CLIENT] Register: POST $url');
    print('[API_CLIENT] Register payload: client_id=$clientId');
    try {
      final response = await http.post(
        url,
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({'client_id': clientId}),
      );
      print('[API_CLIENT] Register response: ${response.statusCode}');
      print('[API_CLIENT] Register body: ${response.body}');

      if (response.statusCode == 200) {
        return jsonDecode(response.body) as Map<String, dynamic>;
      } else {
        throw Exception(
          'Registration failed: ${response.statusCode} - ${response.body}',
        );
      }
    } catch (e, stackTrace) {
      print('[API_CLIENT_ERROR] Registration failed: $e');
      print('[API_CLIENT_ERROR] Stack trace: $stackTrace');
      rethrow;
    }
  }

  /// Fetch global model parameters from server
  Future<Map<String, dynamic>> fetchGlobalParams() async {
    final url = Uri.parse('$_baseUrl/global-params-json');
    print('[API_CLIENT] Fetch global params: GET $url');
    try {
      final response = await http.get(url);
      print(
        '[API_CLIENT] Fetch global params response: ${response.statusCode}',
      );
      print('[API_CLIENT] Response body length: ${response.body.length}');

      if (response.statusCode == 200) {
        final data = jsonDecode(response.body) as Map<String, dynamic>;
        print('[API_CLIENT] Model config: ${data['model_config']}');
        print(
          '[API_CLIENT] Params count: ${(data['params'] as List?)?.length ?? 'N/A'}',
        );
        return data;
      } else if (response.statusCode == 404) {
        // Model not initialized on server
        print('[API_CLIENT_ERROR] Model not initialized (404)');
        throw Exception(
          'Model not initialized on server. Please initialize the model first:\n\n'
          'Run on your PC: curl -X POST "$_baseUrl/init-model?num_users=943&num_items=1682&embedding_dim=16
"\n\n'
          'Or use the API docs at: $_baseUrl/docs',
        );
      } else {
        throw Exception(
          'Failed to fetch global params: ${response.statusCode} - ${response.body}',
        );
      }
    } catch (e, stackTrace) {
```

```
      print('[API_CLIENT_ERROR] Fetch global params failed: $e');
      print('[API_CLIENT_ERROR] Stack trace: $stackTrace');
      rethrow;
    }
  }

  /// Upload client model parameters to server
  Future<Map<String, dynamic>> uploadParams({
    required String clientId,
    required List<Map<String, dynamic>> params,
    required int sampleCount,
  }) async {
    final url = Uri.parse('$_baseUrl/upload-params-json');
    final payload = {
      'client_id': clientId,
      'params': params,
      'sample_count': sampleCount,
    };
    print('[API_CLIENT] Upload params: POST $url');
    print(
      '[API_CLIENT] Upload payload: client_id=$clientId, sample_count=$sampleCount, params_count=${params.l
ength}',
    );
    try {
      final response = await http.post(
        url,
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode(payload),
      );
      print('[API_CLIENT] Upload response: ${response.statusCode}');
      print('[API_CLIENT] Upload body: ${response.body}');

      if (response.statusCode == 200) {
        return jsonDecode(response.body) as Map<String, dynamic>;
      } else {
        throw Exception(
          'Upload failed: ${response.statusCode} - ${response.body}',
        );
      }
    } catch (e, stackTrace) {
      print('[API_CLIENT_ERROR] Upload failed: $e');
      print('[API_CLIENT_ERROR] Stack trace: $stackTrace');
      rethrow;
    }
  }

  /// Upload resource metrics (CPU, memory, battery, etc.)
  Future<void> uploadResourceMetrics({
    required String clientId,
    required Map<String, dynamic> metrics,
  }) async {
    // This endpoint should be added to server.py
    // For now, we'll include it in upload-params or create a separate endpoint
    // Implementation depends on server capability
  }

  /// Upload mobile experiment results to server (saves to PC)
  Future<Map<String, dynamic>> uploadMobileResults({
    required String experimentId,
    required Map<String, dynamic> experimentData,
  }) async {
    final url = Uri.parse('$_baseUrl/upload-mobile-results');
    final payload = {
      'experiment_id': experimentId,
      'experiment_data': experimentData,
    };

    print('[API_CLIENT] Upload mobile results: POST $url');
    print('[API_CLIENT] Experiment ID: $experimentId');

    try {
      final response = await http.post(
        url,
        headers: {'Content-Type': 'application/json'},
```

```
        body: jsonEncode(payload),
      );

      print('[API_CLIENT] Upload results response: ${response.statusCode}');

      if (response.statusCode == 200) {
        return jsonDecode(response.body) as Map<String, dynamic>;
      } else {
        throw Exception(
          'Upload failed: ${response.statusCode} - ${response.body}',
        );
      }
    } catch (e, stackTrace) {
      print('[API_CLIENT_ERROR] Upload results failed: $e');
      print('[API_CLIENT_ERROR] Stack trace: $stackTrace');
      rethrow;
    }
  }

  /// Send recommendation feedback (e.g., mark item as seen) to server.
  Future<Map<String, dynamic>> sendRecommendationSeenEvent({
    required String clientId,
    required int userId,
    required int itemId,
    double rating = 1.0,
  }) async {
    final url = Uri.parse('$_baseUrl/recommendation-events');
    final payload = {
      'client_id': clientId,
      'user_id': userId,
      'item_id': itemId,
      'rating': rating,
    };

    print('[API_CLIENT] Recommendation event: POST $url');
    print('[API_CLIENT] Recommendation payload: $payload');

    try {
      final response = await http
          .post(
            url,
            headers: {'Content-Type': 'application/json'},
            body: jsonEncode(payload),
          )
          .timeout(const Duration(seconds: 10));

      print(
        '[API_CLIENT] Recommendation event response: ${response.statusCode}',
      );
      print('[API_CLIENT] Recommendation event body: ${response.body}');

      if (response.statusCode == 200) {
        return jsonDecode(response.body) as Map<String, dynamic>;
      }
      throw Exception(
        'Recommendation event failed: ${response.statusCode} - ${response.body}',
      );
    } catch (e, stackTrace) {
      print('[API_CLIENT_ERROR] Recommendation event failed: $e');
      print('[API_CLIENT_ERROR] Stack trace: $stackTrace');
      rethrow;
    }
  }
}
```

File: federated_learning_in_mobile/lib/services/fl_client.dart

```
import '../models/matrix_factorization.dart';
import '../utils/model_encoder.dart';
import '../utils/resource_monitor.dart';
import 'api_client.dart';

/// Federated Learning Client for mobile devices
class FederatedLearningClient {
```

```
  final ApiClient apiClient;
  final ResourceMonitor resourceMonitor;

  MatrixFactorization? model;
  String clientId;
  String serverUrl;

  int numUsers;
  int numItems;
  int embeddingDim;

  // Training configuration
  double learningRate = 0.01;
  int localEpochs = 1;
  int batchSize = 32;

  // Local training data: List of (user_id, item_id, rating) tuples
  List<List<dynamic>> localData = [];

  FederatedLearningClient({
    required this.clientId,
    required this.serverUrl,
    required this.numUsers,
    required this.numItems,
    this.embeddingDim = 16,
  }) : apiClient = ApiClient(serverUrl: serverUrl),
       resourceMonitor = ResourceMonitor();

  /// Initialize client (check server, register, initialize monitoring)
  Future<void> initialize() async {
    print('[FL_CLIENT] Initializing client: $clientId');
    try {
      // Check server health
      print('[FL_CLIENT] Checking server health...');
      await apiClient.healthCheck();
      print('[FL_CLIENT] Server health check passed');

      // Register with server
      print('[FL_CLIENT] Registering client...');
      await apiClient.register(clientId);
      print('[FL_CLIENT] Client registered successfully');

      // Initialize resource monitoring
      print('[FL_CLIENT] Initializing resource monitoring...');
      try {
        await resourceMonitor.initialize();
        print('[FL_CLIENT] Resource monitoring initialized');
      } catch (e) {
        print('[FL_CLIENT_WARN] Resource monitoring unavailable: $e');
      }
    } catch (e, stackTrace) {
      print('[FL_CLIENT_ERROR] Initialization failed: $e');
      print('[FL_CLIENT_ERROR] Stack trace: $stackTrace');
      rethrow;
    }
  }

  /// Load local training data
  void loadLocalData(List<List<dynamic>> data) {
    localData = data; // data: [[user_id, item_id, rating], ...]
  }

  /// Check if local data already contains a user-item interaction.
  bool hasInteraction({required int userId, required int itemId}) {
    for (final sample in localData) {
      if (sample.length < 2) {
        continue;
      }
      final sampleUserId = sample[0];
      final sampleItemId = sample[1];
      if (sampleUserId == userId && sampleItemId == itemId) {
        return true;
      }
    }
```

```
    return false;
  }

  /// Add a seen interaction to local data if it does not exist yet.
  bool addSeenInteraction({
    required int userId,
    required int itemId,
    double rating = 1.0,
  }) {
    if (userId < 0 || userId >= numUsers || itemId < 0 || itemId >= numItems) {
      return false;
    }
    if (hasInteraction(userId: userId, itemId: itemId)) {
      return false;
    }
    localData.add([userId, itemId, rating]);
    return true;
  }

  /// Mark recommendation as seen and upload feedback event to server.
  Future<Map<String, dynamic>> markRecommendationSeen({
    required int userId,
    required int itemId,
    double rating = 1.0,
  }) async {
    addSeenInteraction(userId: userId, itemId: itemId, rating: rating);
    return apiClient.sendRecommendationSeenEvent(
      clientId: clientId,
      userId: userId,
      itemId: itemId,
      rating: rating,
    );
  }

  /// Fetch global model from server
  Future<void> fetchGlobalModel() async {
    print('[FL_CLIENT] Fetching global model...');
    try {
      final response = await apiClient.fetchGlobalParams();

      // Extract model config
      final modelConfig = response['model_config'] as Map<String, dynamic>;
      numUsers = modelConfig['num_users'] as int;
      numItems = modelConfig['num_items'] as int;
      embeddingDim = modelConfig['embedding_dim'] as int;
      print(
        '[FL_CLIENT] Model config: numUsers=$numUsers, numItems=$numItems, embeddingDim=$embeddingDim',
      );

      // Decode parameters
      final paramsJson = response['params'] as List;
      print('[FL_CLIENT] Decoding ${paramsJson.length} parameter tensors...');
      final stateDict = ModelEncoder.jsonParamsToModel(
        paramsJson.map((p) => p as Map<String, dynamic>).toList(),
      );
      print('[FL_CLIENT] Parameters decoded successfully');

      // Create and load model
      print('[FL_CLIENT] Creating model instance...');
      model = MatrixFactorization(
        numUsers: numUsers,
        numItems: numItems,
        embeddingDim: embeddingDim,
      );
      model!.loadStateDict(stateDict);
      print('[FL_CLIENT] Model loaded successfully');
    } catch (e, stackTrace) {
      print('[FL_CLIENT_ERROR] Fetch global model failed: $e');
      print('[FL_CLIENT_ERROR] Stack trace: $stackTrace');
      rethrow;
    }
  }

  /// Train model locally using local data
```

```
  Future<Map<String, dynamic>> trainLocal() async {
    print('[FL_CLIENT] Starting local training...');
    if (model == null) {
      print('[FL_CLIENT_ERROR] Model not initialized');
      throw StateError('Model not initialized. Call fetchGlobalModel() first.');
    }

    if (localData.isEmpty) {
      print('[FL_CLIENT] No local data available, returning empty metrics');
      return {'loss': 0.0, 'samples': 0, 'epochs': 0};
    }

    print(
      '[FL_CLIENT] Training with ${localData.length} samples, $localEpochs epochs, batch size $batchSize',
    );

    final startTime = DateTime.now();
    double totalLoss = 0.0;
    int numBatches = 0;

    // Simple SGD training (one epoch for mobile efficiency)
    for (int epoch = 0; epoch < localEpochs; epoch++) {
      // Shuffle data
      localData.shuffle();

      // Process in batches
      for (int i = 0; i < localData.length; i += batchSize) {
        final batchEnd = (i + batchSize < localData.length)
            ? i + batchSize
            : localData.length;
        final batch = localData.sublist(i, batchEnd);

        // Compute gradients (simplified - full implementation would use proper backprop)
        double batchLoss = 0.0;

        // Initialize gradients
        final userGrad = List.generate(
          numUsers,
          (_) => List.filled(embeddingDim, 0.0),
        );
        final itemGrad = List.generate(
          numItems,
          (_) => List.filled(embeddingDim, 0.0),
        );

        // Compute gradients for batch
        for (var sample in batch) {
          final userId = sample[0] as int;
          final itemId = sample[1] as int;
          final rating = (sample[2] as num).toDouble();

          // Validate IDs are in range before accessing embeddings
          if (userId < 0 ||
              userId >= numUsers ||
              itemId < 0 ||
              itemId >= numItems) {
            print(
              '[FL_CLIENT_ERROR] Invalid IDs in sample: user=$userId (valid: 0-${numUsers - 1}), item=$item
Id (valid: 0-${numItems - 1})',
            );
            continue; // Skip invalid samples
          }

          // Forward pass
          final prediction = model!.predict(userId, itemId);
          final error = prediction - rating;
          batchLoss += error * error;

          // Backward pass (simplified gradient)
          final userEmb = model!.userEmbeddings[userId];
          final itemEmb = model!.itemEmbeddings[itemId];

          for (int j = 0; j < embeddingDim; j++) {
            userGrad[userId][j] += error * itemEmb[j];
```

```
          itemGrad[itemId][j] += error * userEmb[j];
        }
      }

      // Average gradients
      final actualBatchSize = batch.length;
      for (int u = 0; u < numUsers; u++) {
        for (int j = 0; j < embeddingDim; j++) {
          userGrad[u][j] /= actualBatchSize;
        }
      }
      for (int i = 0; i < numItems; i++) {
        for (int j = 0; j < embeddingDim; j++) {
          itemGrad[i][j] /= actualBatchSize;
        }
      }

      // Update parameters
      model!.updateParameters(userGrad, itemGrad, learningRate);

      totalLoss += batchLoss / actualBatchSize;
      numBatches++;
    }
  }

  final trainingTime = DateTime.now().difference(startTime).inMilliseconds;
  final avgLoss = numBatches > 0 ? totalLoss / numBatches : 0.0;

  print(
    '[FL_CLIENT] Training complete: loss=$avgLoss, samples=${localData.length}, time=${trainingTime}ms',
  );

  return {
    'loss': avgLoss,
    'samples': localData.length,
    'epochs': localEpochs,
    'training_time_ms': trainingTime,
  };
}

/// Upload local model parameters to server
Future<Map<String, dynamic>> uploadParams() async {
  print('[FL_CLIENT] Uploading parameters...');
  if (model == null) {
    print('[FL_CLIENT_ERROR] Model not initialized');
    throw StateError('Model not initialized');
  }

  print('[FL_CLIENT] Getting model state dict...');
  final stateDict = model!.getStateDict();
  print('[FL_CLIENT] Encoding parameters to JSON...');
  final paramsJson = ModelEncoder.modelToJsonParams(stateDict);
  print('[FL_CLIENT] Encoded ${paramsJson.length} parameter tensors');

  // Get resource metrics
  print('[FL_CLIENT] Getting resource metrics...');
  final metrics = await resourceMonitor.getMetrics();
  print('[FL_CLIENT] Resource metrics: $metrics');

  print('[FL_CLIENT] Uploading to server...');
  final response = await apiClient.uploadParams(
    clientId: clientId,
    params: paramsJson,
    sampleCount: localData.length,
  );
  print('[FL_CLIENT] Upload successful');

  return {...response, 'resource_metrics': metrics};
}

/// Evaluate model on local data (used as test set proxy)
Map<String, dynamic> evaluateModel() {
  print('[FL_CLIENT] Evaluating model on local data...');
  if (model == null) {
```

```
    print('[FL_CLIENT_ERROR] Model not initialized');
    return {
      'accuracy': 0.0,
      'mse': 0.0,
      'mae': 0.0,
      'Hit@10': 0.0,
      'NDCG@10': 0.0,
      'Precision@10': 0.0,
      'Recall@10': 0.0,
    };
  }

  if (localData.isEmpty) {
    print('[FL_CLIENT] No local data for evaluation');
    return {
      'accuracy': 0.0,
      'mse': 0.0,
      'mae': 0.0,
      'Hit@10': 0.0,
      'NDCG@10': 0.0,
      'Precision@10': 0.0,
      'Recall@10': 0.0,
    };
  }

  double totalLoss = 0.0;
  double totalMae = 0.0;
  int correctPredictions = 0;
  int totalSamples = 0;

  // Evaluate on local data
  for (var sample in localData) {
    final userId = sample[0] as int;
    final itemId = sample[1] as int;
    final rating = (sample[2] as num).toDouble();

    // Validate IDs
    if (userId < 0 ||
        userId >= numUsers ||
        itemId < 0 ||
        itemId >= numItems) {
      continue;
    }

    try {
      final prediction = model!.predict(userId, itemId);
      final error = prediction - rating;

      totalLoss += error * error; // MSE
      totalMae += error.abs();

      // Binary classification accuracy (threshold at 0.5)
      final predBinary = prediction > 0.5 ? 1.0 : 0.0;
      if (predBinary == rating) {
        correctPredictions++;
      }
      totalSamples++;
    } catch (e) {
      print(
        '[FL_CLIENT_ERROR] Evaluation error for user=$userId, item=$itemId: $e',
      );
    }
  }

  if (totalSamples == 0) {
    return {
      'accuracy': 0.0,
      'mse': 0.0,
      'mae': 0.0,
      'Hit@10': 0.0,
      'NDCG@10': 0.0,
      'Precision@10': 0.0,
      'Recall@10': 0.0,
    };
```

```
    }

    final mse = totalLoss / totalSamples;
    final mae = totalMae / totalSamples;
    final accuracy = correctPredictions / totalSamples;

    // For recommendation metrics, we'll use simplified versions
    // Since we don't have a full test set, we compute basic metrics
    final hitAt10 = accuracy; // Simplified: using accuracy as proxy
    final ndcgAt10 = accuracy; // Simplified
    final precisionAt10 = accuracy;
    final recallAt10 = accuracy;

    print(
      '[FL_CLIENT] Evaluation complete: accuracy=$accuracy, mse=$mse, mae=$mae',
    );

    return {
      'accuracy': accuracy,
      'mse': mse,
      'mae': mae,
      'Hit@10': hitAt10,
      'NDCG@10': ndcgAt10,
      'Precision@10': precisionAt10,
      'Recall@10': recallAt10,
      'samples': totalSamples,
    };
  }

  /// Execute one complete federated learning round
  Future<Map<String, dynamic>> runTrainingRound() async {
    print('[FL_CLIENT] ========== Starting training round ==========');
    try {
      // 1. Fetch global model (will throw if not initialized)
      print('[FL_CLIENT] Step 1/3: Fetching global model...');
      await fetchGlobalModel();

      // 2. Train locally
      print('[FL_CLIENT] Step 2/3: Training locally...');
      final trainMetrics = await trainLocal();

      // 2.5. Evaluate model after training (before uploading)
      print('[FL_CLIENT] Step 2.5/3: Evaluating model...');
      final testMetrics = evaluateModel();

      // 3. Upload parameters
      print('[FL_CLIENT] Step 3/3: Uploading parameters...');
      final uploadMetrics = await uploadParams();

      print('[FL_CLIENT] ========== Training round complete ==========');
      return {
        'train': trainMetrics,
        'test': testMetrics,
        'upload': uploadMetrics,
      };
    } catch (e, stackTrace) {
      print('[FL_CLIENT_ERROR] Training round failed: $e');
      print('[FL_CLIENT_ERROR] Stack trace: $stackTrace');
      rethrow;
    }
  }
}
```

File: federated_learning_in_mobile/lib/services/movie_title_service.dart

```
import 'package:flutter/services.dart' show rootBundle;

class MovieTitleService {
  const MovieTitleService();

  static const String _assetPath = 'assets/movielens_titles.csv';

  Future<Map<int, String>> loadTitles() async {
    final csvData = await rootBundle.loadString(_assetPath);
```

```
    final titles = <int, String>{};

    for (final rawLine in csvData.split('\n')) {
      final line = rawLine.trim();
      if (line.isEmpty || line.startsWith('item_id')) {
        continue;
      }

      final commaIndex = line.indexOf(',');
      if (commaIndex <= 0) {
        continue;
      }

      final idText = line.substring(0, commaIndex).trim();
      final itemId = int.tryParse(idText);
      if (itemId == null) {
        continue;
      }

      var title = line.substring(commaIndex + 1).trim();
      if (title.startsWith('"') && title.endsWith('"') && title.length >= 2) {
        title = title.substring(1, title.length - 1).replaceAll('""', '"');
      }

      titles[itemId] = title;
    }

    return titles;
  }
}
```

File: federated_learning_in_mobile/lib/services/recommendation_service.dart

---

```
import 'dart:math';

import 'fl_client.dart';

class MovieRecommendation {
  final int itemId;
  final String title;
  final double score;
  final bool isSeen;
  final String explanation;

  const MovieRecommendation({
    required this.itemId,
    required this.title,
    required this.score,
    required this.isSeen,
    required this.explanation,
  });

  MovieRecommendation copyWith({
    int? itemId,
    String? title,
    double? score,
    bool? isSeen,
    String? explanation,
  }) {
    return MovieRecommendation(
      itemId: itemId ?? this.itemId,
      title: title ?? this.title,
      score: score ?? this.score,
      isSeen: isSeen ?? this.isSeen,
      explanation: explanation ?? this.explanation,
    );
  }
}

class RecommendationService {
  const RecommendationService();

  List<int> getCandidateUsers(FederatedLearningClient client) {
    final ids =
```

```
      client.localData
          .map((sample) => sample[0])
          .whereType<int>()
          .where((id) => id >= 0 && id < client.numUsers)
          .toSet()
          .toList()
        ..sort();

  if (ids.isNotEmpty) {
    return ids;
  }

  if (client.numUsers > 0) {
    return [0];
  }

  return <int>[];
}

List<MovieRecommendation> topNRecommendations({
  required FederatedLearningClient client,
  required int userId,
  required int topN,
  Map<int, String> movieTitles = const <int, String>{},
  bool excludeSeenItems = true,
}) {
  if (client.model == null) {
    throw StateError('Model is not loaded.');
  }
  if (client.numItems <= 0 || client.numUsers <= 0) {
    return const <MovieRecommendation>[];
  }
  if (userId < 0 || userId >= client.numUsers) {
    throw ArgumentError('Invalid user id: $userId');
  }

  final seenItems = <int>{};
  for (final sample in client.localData) {
    if (sample.length < 2) {
      continue;
    }
    final sampleUserId = sample[0];
    final sampleItemId = sample[1];
    if (sampleUserId is int &&
        sampleItemId is int &&
        sampleUserId == userId) {
      seenItems.add(sampleItemId);
    }
  }

  final recommendations = <MovieRecommendation>[];
  for (int itemId = 0; itemId < client.numItems; itemId++) {
    final isSeen = seenItems.contains(itemId);
    if (excludeSeenItems && isSeen) {
      continue;
    }

    final score = client.model!.predict(userId, itemId);
    final movieTitle = movieTitles[itemId] ?? 'Movie #$itemId';
    final seenLabel = isSeen ? 'Seen before' : 'New for this user';
    final explanation =
        'Predicted score ${score.toStringAsFixed(4)} - $seenLabel';

    recommendations.add(
      MovieRecommendation(
        itemId: itemId,
        title: movieTitle,
        score: score,
        isSeen: isSeen,
        explanation: explanation,
      ),
    );
  }
```

```
    recommendations.sort((a, b) => b.score.compareTo(a.score));
    return recommendations.take(max(0, topN)).toList(growable: false);
  }
}
```

File: federated_learning_in_mobile/lib/utils/metrics_collector.dart

---

```
import 'dart:convert';
import 'dart:io';
import 'package:path_provider/path_provider.dart';
import 'package:csv/csv.dart';
import 'package:flutter/foundation.dart';
import 'package:path/path.dart' as path;

/// Metrics Collector for Federated Learning Experiments (Mobile)
/// Collects and saves all experimental metrics to JSON/CSV
class MetricsCollector {
  final String experimentId;
  late Directory resultsDir;

  // Store all collected metrics
  final Map<String, dynamic> experimentData;

  MetricsCollector({
    required this.experimentId,
  }) : experimentData = {
          'experiment_id': experimentId,
          'timestamp': DateTime.now().toIso8601String(),
          'config': <String, dynamic>{},
          'rounds': <Map<String, dynamic>>[],
          'final_metrics': <String, dynamic>{},
          'client_metrics': <Map<String, dynamic>>[],
          'mobile_metrics': <Map<String, dynamic>>[],
        };

  /// Initialize the results directory - saves to Downloads folder for easy access
  Future<void> initialize() async {
    // Check if running on web (not supported)
    try {
      if (kIsWeb) {
        throw UnsupportedError('Metrics collection not supported on web');
      }
    } catch (e) {
      // kIsWeb might not be available, continue anyway
    }

    // Try to use external storage Downloads folder (most accessible)
    try {
      final externalDir = await getExternalStorageDirectory();
      if (externalDir != null) {
        // Navigate to Downloads folder
        // Android external storage structure: /storage/emulated/0/Download
        final downloadsPath = Platform.isAndroid
            ? '/storage/emulated/0/Download/FL_Results'
            : '${externalDir.path}/Downloads/FL_Results';

        // Try the standard Downloads path first
        final downloadsDir = Directory(downloadsPath);

        // If that doesn't exist, try alternative paths
        if (!await downloadsDir.exists() && Platform.isAndroid) {
          // Alternative: Use external storage root + Download
          final altPath = path.join(externalDir.parent.path, 'Download', 'FL_Results');
          resultsDir = Directory(altPath);
        } else {
          resultsDir = downloadsDir;
        }

        if (!await resultsDir.exists()) {
          await resultsDir.create(recursive: true);
        }

        print('[METRICS_COLLECTOR] Using Downloads folder: ${resultsDir.path}');
        return;
```

```
    }
  } catch (e) {
    print('[METRICS_COLLECTOR] Warning: Downloads folder failed: $e');
  }

  // Fallback: Try external storage directory
  try {
    final externalDir = await getExternalStorageDirectory();
    if (externalDir != null) {
      resultsDir = Directory('${externalDir.path}/FL_Results');
      if (!await resultsDir.exists()) {
        await resultsDir.create(recursive: true);
      }
      print('[METRICS_COLLECTOR] Using external storage: ${resultsDir.path}');
      return;
    }
  } catch (e) {
    print('[METRICS_COLLECTOR] Warning: External storage failed: $e');
  }

  // Fallback: Application documents directory
  try {
    final appDocDir = await getApplicationDocumentsDirectory();
    resultsDir = Directory('${appDocDir.path}/FL_Results');

    if (!await resultsDir.exists()) {
      await resultsDir.create(recursive: true);
    }

    print('[METRICS_COLLECTOR] Using app documents: ${resultsDir.path}');
    return;
  } catch (e) {
    print('[METRICS_COLLECTOR] Warning: Application documents directory failed: $e');
  }

  // Last resort: temporary directory
  try {
    final tempDir = await getTemporaryDirectory();
    resultsDir = Directory('${tempDir.path}/FL_Results');
    if (!await resultsDir.exists()) {
      await resultsDir.create(recursive: true);
    }
    print('[METRICS_COLLECTOR] Using temporary directory: ${resultsDir.path}');
  } catch (e) {
    throw Exception('Could not initialize any storage directory: $e');
  }
}

/// Get the results directory path (for display)
String get resultsPath => resultsDir.path;

/// Get the full experiment data as a Map (for uploading to server)
Map<String, dynamic> getExperimentData() {
  return Map<String, dynamic>.from(experimentData);
}

/// Get list of all result files
Future<List<File>> getResultFiles() async {
  try {
    if (!await resultsDir.exists()) {
      return [];
    }

    final files = resultsDir.listSync()
        .whereType<File>()
        .where((f) => f.path.endsWith('.json') || f.path.endsWith('.csv'))
        .toList();

    return files;
  } catch (e) {
    print('[METRICS_COLLECTOR] Error listing files: $e');
    return [];
  }
}
```

```
/// Set experiment configuration
void setConfig(Map<String, dynamic> config) {
  experimentData['config'] = config;
  print('[METRICS_COLLECTOR] Set experiment config');
}

/// Add metrics for a training round
void addRoundMetrics({
  required int roundNum,
  required double trainLoss,
  Map<String, dynamic>? testMetrics,
  Map<String, dynamic>? aggregationInfo,
  List<Map<String, dynamic>>? clientMetrics,
  Map<String, dynamic>? resourceMetrics,
}) {
  final roundData = <String, dynamic>{
    'round': roundNum,
    'train_loss': trainLoss,
    'test_metrics': testMetrics ?? <String, dynamic>{},
    'aggregation': aggregationInfo ?? <String, dynamic>{},
    'client_metrics': clientMetrics ?? <Map<String, dynamic>>[],
    'resource_metrics': resourceMetrics ?? <String, dynamic>{},
    'timestamp': DateTime.now().toIso8601String(),
  };

  (experimentData['rounds'] as List).add(roundData);
  print('[METRICS_COLLECTOR] Added metrics for round $roundNum');
}

/// Add final evaluation metrics
void addFinalMetrics(Map<String, dynamic> metrics) {
  experimentData['final_metrics'] = metrics;
  print('[METRICS_COLLECTOR] Added final metrics');
}

/// Add per-client metrics
void addClientMetrics(String clientId, Map<String, dynamic> metrics) {
  final clientData = <String, dynamic>{
    'client_id': clientId,
    ...metrics,
    'timestamp': DateTime.now().toIso8601String(),
  };

  (experimentData['client_metrics'] as List).add(clientData);
}

/// Add mobile device resource metrics
void addMobileMetrics(String deviceId, Map<String, dynamic> metrics) {
  final mobileData = <String, dynamic>{
    'device_id': deviceId,
    ...metrics,
    'timestamp': DateTime.now().toIso8601String(),
  };

  (experimentData['mobile_metrics'] as List).add(mobileData);
}

/// Save metrics to JSON file
Future<String> saveJson([String? filename]) async {
  await initialize();

  filename ??= '$experimentId.json';

  final file = File('${resultsDir.path}/$filename');

  // Convert to JSON with proper formatting
  final jsonString = JsonEncoder.withIndent('  ').convert(experimentData);
  await file.writeAsString(jsonString);

  print('[METRICS_COLLECTOR] Saved JSON to: ${file.path}');
  return file.path;
}
```

```
/// Save summary to CSV file
Future<String> saveCsvSummary([String? filename]) async {
  await initialize();

  filename ??= '${experimentId}_summary.csv';

  final file = File('${resultsDir.path}/$filename');

  // Create CSV rows
  final List<List<dynamic>> csvData = [];

  // Header
  csvData.add([
    'Round',
    'Train_Loss',
    'NDCG@10',
    'Hit@10',
    'Precision@10',
    'Recall@10',
    'MSE',
    'MAE',
    'Accuracy',
    'Num_Clients',
    'Total_Samples',
    'Training_Time_MS',
    'Battery_Drain',
  ]);

  // Round data
  final rounds = experimentData['rounds'] as List;
  for (final roundData in rounds) {
    final testMetrics = roundData['test_metrics'] as Map<String, dynamic>? ?? {};
    final agg = roundData['aggregation'] as Map<String, dynamic>? ?? {};
    final resourceMetrics = roundData['resource_metrics'] as Map<String, dynamic>? ?? {};

    csvData.add([
      roundData['round'],
      roundData['train_loss']?.toString() ?? '',
      testMetrics['NDCG@10']?.toString() ?? '',
      testMetrics['Hit@10']?.toString() ?? '',
      testMetrics['Precision@10']?.toString() ?? '',
      testMetrics['Recall@10']?.toString() ?? '',
      testMetrics['mse']?.toString() ?? '',
      testMetrics['mae']?.toString() ?? '',
      testMetrics['accuracy']?.toString() ?? '',
      agg['num_clients']?.toString() ?? '',
      agg['total_samples']?.toString() ?? '',
      resourceMetrics['training_time_ms']?.toString() ?? '',
      resourceMetrics['battery_drain']?.toString() ?? '',
    ]);
  }

  // Final metrics row
  final finalMetrics = experimentData['final_metrics'] as Map<String, dynamic>? ?? {};
  csvData.add([
    'FINAL',
    '',
    finalMetrics['NDCG@10']?.toString() ?? '',
    finalMetrics['Hit@10']?.toString() ?? '',
    finalMetrics['Precision@10']?.toString() ?? '',
    finalMetrics['Recall@10']?.toString() ?? '',
    finalMetrics['mse']?.toString() ?? '',
    finalMetrics['mae']?.toString() ?? '',
    finalMetrics['accuracy']?.toString() ?? '',
    '',
    '',
    '',
    '',
  ]);

  // Write CSV
  final csvString = const ListToCsvConverter().convert(csvData);
  await file.writeAsString(csvString);
```

```
    print('[METRICS_COLLECTOR] Saved CSV summary to: ${file.path}');
    return file.path;
  }

  /// Get summary statistics
  Map<String, dynamic> getSummary() {
    final rounds = experimentData['rounds'] as List;

    if (rounds.isEmpty) {
      return {'message': 'No rounds collected yet'};
    }

    // Extract metrics across rounds
    final trainLosses = rounds
        .map((r) => (r as Map)['train_loss'] as double?)
        .where((loss) => loss != null)
        .cast<double>()
        .toList();

    final ndcgScores = rounds
        .map((r) {
          final testMetrics = (r as Map)['test_metrics'] as Map?;
          return testMetrics?['NDCG@10'] as double?;
        })
        .where((score) => score != null)
        .cast<double>()
        .toList();

    final hitScores = rounds
        .map((r) {
          final testMetrics = (r as Map)['test_metrics'] as Map?;
          return testMetrics?['Hit@10'] as double?;
        })
        .where((score) => score != null)
        .cast<double>()
        .toList();

    return {
      'experiment_id': experimentId,
      'num_rounds': rounds.length,
      'final_train_loss': trainLosses.isNotEmpty ? trainLosses.last : null,
      'final_ndcg@10': ndcgScores.isNotEmpty ? ndcgScores.last : null,
      'final_hit@10': hitScores.isNotEmpty ? hitScores.last : null,
      'best_ndcg@10': ndcgScores.isNotEmpty ? ndcgScores.reduce((a, b) => a > b ? a : b) : null,
      'best_hit@10': hitScores.isNotEmpty ? hitScores.reduce((a, b) => a > b ? a : b) : null,
      'config': experimentData['config'],
    };
  }
}

/// Create standardized experiment ID
String createExperimentId({
  double? dpEpsilon,
  double? alpha,
  int embeddingDim = 16,
  int numClients = 3,
  int? seed,
  String? deviceId,
}) {
  final parts = <String>[];

  if (dpEpsilon == null) {
    parts.add('dp_inf');
  } else {
    parts.add('dp_$dpEpsilon');
  }

  if (alpha != null) {
    parts.add('alpha_$alpha');
  }

  parts.add('dim_$embeddingDim');
  parts.add('clients_$numClients');
```

```
  if (seed != null) {
    parts.add('seed_$seed');
  }

  if (deviceId != null) {
    // Clean device ID for filename
    final cleanId = deviceId.replaceAll(RegExp(r'[^\w-]'), '_');
    parts.add('device_$cleanId');
  }

  // Add timestamp for uniqueness
  final timestamp = DateTime.now().millisecondsSinceEpoch;
  parts.add('t$timestamp');

  return parts.join('_');
}
```

File: federated_learning_in_mobile/lib/utils/model_encoder.dart

```
import 'dart:convert';
import 'package:flutter/foundation.dart';

/// Utilities for encoding/decoding model parameters to/from JSON format
/// Compatible with Python server's portable JSON format

class ModelEncoder {
  /// Convert model parameters to portable JSON format (compatible with server)
  /// Returns list of parameter dictionaries with base64-encoded data
  static List<Map<String, dynamic>> modelToJsonParams(
    Map<String, dynamic> stateDict,
  ) {
    print('[MODEL_ENCODER] Encoding model parameters to JSON...');
    print('[MODEL_ENCODER] State dict keys: ${stateDict.keys.toList()}');
    final params = <Map<String, dynamic>>[];

    stateDict.forEach((name, value) {
      print('[MODEL_ENCODER] Processing parameter: $name');
      if (value is List<List<double>>) {
        // Convert 2D list to flat array and then to base64
        final flatList = <double>[];
        var shape = <int>[];

        if (value.isNotEmpty) {
          shape = [value.length, value[0].length];
          print('[MODEL_ENCODER] Parameter $name shape: $shape');
          for (var row in value) {
            flatList.addAll(row);
          }
        } else {
          print('[MODEL_ENCODER] Warning: Parameter $name is empty');
        }

        // Convert to Float32 bytes (4 bytes per float)
        final bytes = <int>[];
        for (var val in flatList) {
          // Convert double to Float32 bytes (simplified - actual implementation
          // should use proper Float32 encoding)
          final bytes32 = _doubleToFloat32Bytes(val);
          bytes.addAll(bytes32);
        }

        print('[MODEL_ENCODER] Parameter $name: ${flatList.length} values, ${bytes.length} bytes');

        // Encode to base64
        final base64Data = base64Encode(bytes);
        print('[MODEL_ENCODER] Parameter $name base64 length: ${base64Data.length}');

        params.add({
          'name': name,
          'shape': shape,
          'dtype': 'float32',
          'data': base64Data,
        });
```

```
      } else {
        print('[MODEL_ENCODER] Warning: Parameter $name has unsupported type: ${value.runtimeType}');
      }
    });

    print('[MODEL_ENCODER] Encoded ${params.length} parameters');
    return params;
  }

  /// Convert Float32 bytes (4 bytes) from double
  static List<int> _doubleToFloat32Bytes(double value) {
    // Simplified: convert double to 4-byte representation
    // For production, use proper Float32 encoding
    final bytes = ByteData(4);
    bytes.setFloat32(0, value.toDouble(), Endian.little);
    return bytes.buffer.asUint8List().toList();
  }

  /// Decode JSON parameters into model state dictionary
  static Map<String, dynamic> jsonParamsToModel(List<Map<String, dynamic>> params) {
    print('[MODEL_ENCODER] Decoding ${params.length} parameters from JSON...');
    final stateDict = <String, dynamic>{};

    for (var param in params) {
      try {
        final name = param['name'] as String;
        final shape = List<int>.from(param['shape'] as List);
        final dtype = param['dtype'] as String;
        final base64Data = param['data'] as String;

        print('[MODEL_ENCODER] Decoding parameter: $name, shape: $shape, dtype: $dtype');
        print('[MODEL_ENCODER] Base64 data length: ${base64Data.length}');

        if (dtype == 'float32') {
          // Decode base64
          final bytes = base64Decode(base64Data);
          print('[MODEL_ENCODER] Decoded bytes length: ${bytes.length}');

          // Convert bytes to Float32 values
          final values = <double>[];
          for (int i = 0; i < bytes.length; i += 4) {
            if (i + 4 <= bytes.length) {
              final byteData = ByteData.view(
                bytes.buffer,
                bytes.offsetInBytes + i,
                4,
              );
              values.add(byteData.getFloat32(0, Endian.little));
            }
          }

          print('[MODEL_ENCODER] Decoded ${values.length} float32 values');

          // Reshape to 2D list
          if (shape.length == 2) {
            final matrix = <List<double>>[];
            for (int i = 0; i < shape[0]; i++) {
              final row = <double>[];
              for (int j = 0; j < shape[1]; j++) {
                final idx = i * shape[1] + j;
                if (idx < values.length) {
                  row.add(values[idx]);
                } else {
                  print('[MODEL_ENCODER] Warning: Index out of bounds for $name at [$i, $j]');
                }
              }
              matrix.add(row);
            }
            stateDict[name] = matrix;
            print('[MODEL_ENCODER] Successfully decoded parameter $name: ${matrix.length}x${matrix.isNotEmp
ty ? matrix[0].length : 0}');
          } else {
            print('[MODEL_ENCODER] Warning: Parameter $name has unsupported shape length: ${shape.length}')
;
```

```
          }
        } else {
          print('[MODEL_ENCODER] Warning: Parameter $name has unsupported dtype: $dtype');
        }
      } catch (e, stackTrace) {
        print('[MODEL_ENCODER_ERROR] Failed to decode parameter: $e');
        print('[MODEL_ENCODER_ERROR] Stack trace: $stackTrace');
        rethrow;
      }
    }

    print('[MODEL_ENCODER] Decoded ${stateDict.length} parameters');
    return stateDict;
  }
}
```

File: federated_learning_in_mobile/lib/utils/resource_monitor.dart

---

```
import 'dart:io';
import 'package:battery_plus/battery_plus.dart';
import 'package:device_info_plus/device_info_plus.dart';

/// Resource monitoring utilities for mobile devices
/// Tracks CPU, memory, battery, and network usage

class ResourceMonitor {
  final Battery _battery = Battery();
  final DeviceInfoPlugin _deviceInfo = DeviceInfoPlugin();

  int? _initialBatteryLevel;
  DateTime? _startTime;

  /// Initialize monitoring (record initial state)
  Future<void> initialize() async {
    print('[RESOURCE_MONITOR] Initializing resource monitoring...');
    try {
      _startTime = DateTime.now();
      _initialBatteryLevel = await _battery.batteryLevel;
      print('[RESOURCE_MONITOR] Initial battery level: $_initialBatteryLevel%');
      print('[RESOURCE_MONITOR] Start time: $_startTime');
    } catch (e, stackTrace) {
      print('[RESOURCE_MONITOR_ERROR] Initialization failed: $e');
      print('[RESOURCE_MONITOR_ERROR] Stack trace: $stackTrace');
      rethrow;
    }
  }

  /// Get current battery level
  Future<int> getBatteryLevel() async {
    return await _battery.batteryLevel;
  }

  /// Calculate battery drain since initialization
  Future<double> getBatteryDrain() async {
    if (_initialBatteryLevel == null) {
      await initialize();
    }

    final currentLevel = await _battery.batteryLevel;
    return (_initialBatteryLevel! - currentLevel).toDouble();
  }

  /// Get device information
  Future<Map<String, dynamic>> getDeviceInfo() async {
    if (Platform.isAndroid) {
      final androidInfo = await _deviceInfo.androidInfo;
      return {
        'platform': 'Android',
        'model': androidInfo.model,
        'manufacturer': androidInfo.manufacturer,
        'version': androidInfo.version.release,
        'sdkInt': androidInfo.version.sdkInt,
      };
```

```
    } else if (Platform.isIOS) {
      final iosInfo = await _deviceInfo.iosInfo;
      return {
        'platform': 'iOS',
        'model': iosInfo.model,
        'name': iosInfo.name,
        'systemVersion': iosInfo.systemVersion,
      };
    }
    return {'platform': 'Unknown'};
  }

  /// Get resource metrics (for reporting)
  Future<Map<String, dynamic>> getMetrics() async {
    print('[RESOURCE_MONITOR] Getting resource metrics...');
    try {
      final batteryLevel = await getBatteryLevel();
      final batteryDrain = await getBatteryDrain();
      final deviceInfo = await getDeviceInfo();

      final elapsedSeconds = _startTime != null
          ? DateTime.now().difference(_startTime!).inSeconds
          : 0;

      final metrics = {
        'battery_level': batteryLevel,
        'battery_drain': batteryDrain,
        'elapsed_seconds': elapsedSeconds,
        'device_info': deviceInfo,
        'timestamp': DateTime.now().toIso8601String(),
        // Note: CPU and memory monitoring requires platform channels
        // For thesis, you can add estimates or use platform-specific packages
        'cpu_percent': _estimateCpuUsage(), // Placeholder
        'memory_mb': _estimateMemoryUsage(), // Placeholder
      };

      print(
        '[RESOURCE_MONITOR] Metrics collected: battery=$batteryLevel%, drain=$batteryDrain%, elapsed=${elap
sedSeconds}s',
      );
      return metrics;
    } catch (e, stackTrace) {
      print('[RESOURCE_MONITOR_ERROR] Failed to get metrics: $e');
      print('[RESOURCE_MONITOR_ERROR] Stack trace: $stackTrace');
      rethrow;
    }
  }

  /// Estimate CPU usage (placeholder - requires platform channels for real measurement)
  double _estimateCpuUsage() {
    // For thesis, you can implement platform channels or use profiling tools
    // This is a placeholder
    return 15.0; // Estimated percentage
  }

  /// Estimate memory usage (placeholder)
  double _estimateMemoryUsage() {
    // Requires platform channels for accurate measurement
    // For thesis, you can add actual implementation using dart:ffi or plugins
    return 50.0; // Estimated MB
  }

  /// Measure network bandwidth for data transfer
  static int measureNetworkBytes(
    int bytesTransferred,
    DateTime start,
    DateTime end,
  ) {
    final duration = end.difference(start);
    final bytesPerSecond = duration.inMilliseconds > 0
        ? (bytesTransferred / (duration.inMilliseconds / 1000)).round()
        : 0;
    return bytesPerSecond;
  }
```

```
}
```

File: federated_learning_in_mobile/test/recommendation_service_test.dart

```
import 'package:flutter_test/flutter_test.dart';

import 'package:federated_learning_in_mobile/models/matrix_factorization.dart';
import 'package:federated_learning_in_mobile/services/fl_client.dart';
import 'package:federated_learning_in_mobile/services/recommendation_service.dart';

void main() {
  test('RecommendationService returns titles and explanation text', () {
    final client = FederatedLearningClient(
      clientId: 'test-client',
      serverUrl: 'http://localhost:8000',
      numUsers: 3,
      numItems: 5,
      embeddingDim: 4,
    );

    client.model = MatrixFactorization(
      numUsers: client.numUsers,
      numItems: client.numItems,
      embeddingDim: client.embeddingDim,
    );

    client.loadLocalData([
      [0, 1, 1.0],
      [0, 3, 1.0],
    ]);

    const service = RecommendationService();
    final results = service.topNRecommendations(
      client: client,
      userId: 0,
      topN: 3,
      movieTitles: const {0: 'Toy Story (1995)', 2: 'Four Rooms (1995)'},
      excludeSeenItems: false,
    );

    expect(results.length, 3);
    expect(results.first.title.isNotEmpty, isTrue);
    expect(results.first.explanation.contains('Predicted score'), isTrue);
  });
}
```

File: federated_learning_in_mobile/test/widget_test.dart

```
// This is a basic Flutter widget test.
//
// To perform an interaction with a widget in your test, use the WidgetTester
// utility in the flutter_test package. For example, you can send tap and scroll
// gestures. You can also use WidgetTester to find child widgets in the widget
// tree, read text, and verify that the values of widget properties are correct.

import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

import 'package:federated_learning_in_mobile/main.dart';

void main() {
  testWidgets('App launches and shows dashboard controls', (
    WidgetTester tester,
  ) async {
    // Build our app and trigger a frame.
    await tester.pumpWidget(const FederatedLearningApp());

    // Verify that the app shows the server URL field
    expect(find.text('Federated Client'), findsOneWidget);
    expect(find.text('SERVER URL'), findsOneWidget);
    expect(find.text('CONNECT TO SERVER'), findsOneWidget);
    expect(
      find.text('Demo Mode: Train and Show Recommendations'),
      findsOneWidget,
```

```
    );
    expect(find.byIcon(Icons.movie), findsOneWidget);
  });
}
```